

AI-Powered Clothing App for Personalized Outfit Recommendation

By

Sia Jin Sheng



FACULTY OF COMPUTING AND
INFORMATION TECHNOLOGY

TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND
TECHNOLOGY
KUALA LUMPUR

ACADEMIC YEAR
2025/26

AI-Powered Clothing App for Personalized Outfit Recommendation

By

Sia Jin Sheng

Supervisor: Ms SITI NADIAH BINTI NAIN

A project report submitted to the

Faculty of Computing and Information Technology

in partial fulfillment of the requirement for the

Bachelor of Information Technology (Honours) in Software Systems Development

Faculty of Computing and Information Technology

Tunku Abdul Rahman University of Management and Technology

Kuala Lumpur

Copyright by Tunku Abdul Rahman University of Management and Technology.

All rights reserved. No part of this project documentation may be reproduced, stored in retrieval system, or transmitted in any form or by any means without prior permission of Tunku Abdul Rahman University of Management and Technology.

Declaration

The project submitted herewith is a result of my own efforts in totality and in every aspect of the project works. All information that has been obtained from other sources had been fully acknowledged. I understand that any plagiarism, cheating or collusion or any sorts constitutes a breach of TAR University rules and regulations and would be subjected to disciplinary actions.



Sia Jin Sheng

Bachelor of Information Technology (Honours) in Software Systems Development

ID: 24WMR16773

Abstract

This project presents the development of an AI-powered clothing mobile application designed to provide personalized outfit recommendations based on a user's height, weight, gender, and preferred fashion style. The project addresses the common challenge faced by online shoppers who struggle to select clothing that fits their personal style and body measurements, often resulting in dissatisfaction and product returns. By integrating machine learning-based recommendation logic with a mobile user interface, the system aims to enhance user confidence during the shopping process and improve product discovery.

The scope of the project includes the construction of a Flutter-based mobile application, a backend service for AI processing, and a structured database storing product, user, and recommendation data. Core features include user authentication, product browsing, cart management, wishlist, order simulation, and AI-generated outfit recommendations tailored to individual user profiles.

The system employs a rule-based filtering model combined with similarity scoring to compute personalized recommendations. Supabase Auth, PostgreSQL database storage, Row Level Security, and REST API communication for the recommendation module were used throughout development. Testing involved functional validation of user flows, UI/UX testing, and evaluation of the AI recommendation accuracy using predefined scenarios.

The results show that the developed application successfully provides relevant outfit suggestions and delivers smooth user interaction. While the AI model is lightweight and suitable for prototype use, future improvements may include implementing a deeper learning model and expanding the dataset for better personalization. Overall, the project demonstrates the feasibility of integrating AI-driven recommendations into a mobile shopping application and provides a solid foundation for future enhancements.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Ms. Siti Nadiah Binti Nain, for her guidance, support, and constructive feedback throughout the duration of this project. Her encouragement and expertise have been invaluable in shaping the direction and quality of this work. I would also like to thank the lecturers and moderators who provided academic insights and support along the way. My heartfelt appreciation goes to my family and friends for their continuous motivation, understanding, and patience during the development of this project. Their support has been instrumental to the successful completion of this work.

Chapter 1: Introduction	7
1.1 Project Objectives	7
1.2 Project Background	8
1.3 Advantages and Contributions	9
1.3.1 Advantages for Consumers	9
1.3.2 Contributions to the Industry	10
1.3.3 Contributions to Research	11
1.4 Project Plan	11
1.5 Project Team and Organization	12
1.6 Chapter Summary and Evaluation	13
Chapter 2: Literature Review	14
2.1 Introduction	14
2.2 Related Studies on Fashion Recommendation Systems	14
2.3 Artificial Intelligence and Machine Learning in Fashion	15
2.4 E-Commerce Application Features and Design	16
2.5 Related Systems Comparison	17
2.6 Summary and Evaluation	19
2.7 Reference	19
Chapter 3: Methodology And Requirements Analysis	21
3.1 Introduction	21
3.2 Methodology	21
3.2.1 Selected Development Model: Agile Development	21
3.2.2 Advantages of Agile for This Project	22
3.3 Requirements Gathering Techniques	23
3.4 Requirements Analysis	24
3.4.1 Use Case Diagram	24
3.4.2 Data Flow Diagram (DFD) – Level 0	25
3.5 Functional and Non-Functional Requirements	26
3.5.1 Functional Requirements	26
3.5.2 Non-Functional Requirements	27
3.6 Chapter Summary and Evaluation	28
Chapter 4: System Design	29
4.1 System Overview / Workflow Design	29
4.1.1 System Workflow Steps	29
4.1.2 Workflow Diagram (Description)	32
4.2 User Interface (UI) / System Interaction Design	33
4.2.1 UI Wireframe Description	33
4.3 Data Design	35
4.3.1 Entity Relationship Diagram (ERD)	35
4.3.2 Class Diagram	37
4.3.3 Database Schema	39
4.4 Software Architecture Design	44
4.4.1 Layered Architecture (revised with links to requirements + clear boundaries)	44
4.4.2 Architecture Diagram	45
4.5 Process Design	46
4.5.1 Login Validation Process (Supabase Auth)	46
4.5.2 AI Recommendation Process	46
4.5.3 Simulated Payment Process	47
4.6 Algorithm / Model Design	48

4.6.1 Algorithm Overview and Justification	48
4.6.2 Model Input Features	49
4.6.3 Model Architecture	49
4.6.4 Activation & Loss Functions	49
4.6.5 Compatibility Scoring Formula	50
4.6.6 Final Prediction Equation	50
4.7 Security Design	51
4.7.1 Security Measures, Risk Addressed, and Ownership	52
4.7.2 Security Validation / Testing Plan (Project 2)	53
4.8 Chapter Summary and Evaluation	54
Chapter 5: Implementation and Testing	55
5.1 Implementation	55
5.1.1 Development of Mobile Application	55

Chapter 1: Introduction

1.1 Project Objectives

The primary goal of this project is to design and develop a mobile fashion e-commerce application that improves upon existing clothing retail apps by providing personalized outfit recommendations. Current applications (e.g., H&M, Zara, Uniqlo, Padini) support browsing and purchasing, but they generally provide limited personalisation that matches a user's body profile and preferred fashion style. This project addresses that gap by integrating an AI-based recommendation component and evaluating its effectiveness.

By the end of this project, the objectives are to:

1. Design and implement a functional mobile fashion e-commerce application (core objective)
Develop a mobile application that supports essential e-commerce workflows, including product browsing by category, product details viewing, secure user authentication, cart management, and checkout simulation (or order placement depending on scope).
Success measure: all core modules are implemented and pass functional testing based on defined test cases.
2. Develop a personalised outfit recommendation module based on user profile inputs (core objective)
Implement an AI-based recommendation approach (e.g., rule-based, machine-learning-based, or hybrid, depending on the final confirmed scope) that generates outfit suggestions using user-provided information such as height, weight, and preferred fashion style (e.g., Japanese, Korean, European).
Success measure: the system can generate recommendations for all supported user profiles and styles, and the recommendation logic produces consistent outputs under tested scenarios.
3. Evaluate the effectiveness of the recommendation feature in improving user experience (core objective)
Conduct user evaluation (e.g., usability testing, questionnaire, or user acceptance testing) to assess whether personalisation reduces the time needed to find suitable items and improves satisfaction compared to non-personalised browsing.
Success measure: user evaluation results show positive acceptance (e.g., average satisfaction/usability score above a chosen threshold such as 3.5/5, or majority of users agreeing that recommendations are helpful).

4. Implement supporting engagement features to enhance platform usability (supporting objective)
Add supporting features such as wishlist functionality, product ratings/reviews, and user profile customisation to improve interaction and realism of the shopping experience.
Success measure: supporting features are implemented and validated through functional testing, but they do not take priority over the recommendation module.
5. Ensure the system supports diverse users and style preferences (supporting objective)
Provide recommendation options that accommodate different body profiles and multiple style categories to promote inclusivity in shopping experiences.
Success measure: the application supports all defined body-profile ranges and style categories listed in the project scope, verified through test data.

1.2 Project Background

The fashion and apparel industry is one of the largest and fastest-growing segments of global e-commerce. In Southeast Asia, the market is expected to generate USD 53.45 billion in revenue by 2025, supported by an annual growth rate of 3.6% (Statista, 2023). This growth is closely linked to mobile-first consumer behaviour, where 89% of internet users access the web primarily via smartphones (MarketResearchSEA, 2023). In addition, 94% of urban internet users in the region engage in e-commerce activities (Daxue Consulting, 2022), indicating that mobile shopping has become a widely adopted purchasing channel.

Despite the rapid expansion of online fashion retail, a key user problem remains unresolved: “many shoppers struggle to choose clothing that fits their body measurements and matches their preferred style because most mobile fashion apps offer limited personalisation”. In physical stores, customers can try on items and quickly judge fit and appearance, but online shopping removes this ability. As a result, users often experience uncertainty when deciding what to buy, which contributes to dissatisfaction, incorrect sizing decisions, and higher product return rates.

This personalisation gap has direct business impact. When users cannot confidently determine suitability, they are more likely to delay purchasing decisions or abandon the shopping process entirely. Studies report that poor user experience is a major contributor to cart abandonment, with 82% of users abandoning carts due to unsatisfactory experiences during online shopping (VWO, 2023). In fashion e-commerce specifically, uncertainty about fit and style suitability increases hesitation, lowering conversion rates and increasing the likelihood of returns. Therefore, improving personalisation is not only a usability enhancement but also a practical approach to reducing abandonment and improving purchase confidence.

Most existing fashion retailer applications (e.g., H&M, Zara, Uniqlo, Padini) primarily replicate catalogue-based browsing by providing product listings, filters, and generic recommendations such as “customers also bought.” While useful, these

recommendations typically rely on general popularity or purchase history and do not sufficiently consider "individual body profile" and "style preference", which are critical factors in apparel selection.

To address this gap, this project proposes a mobile fashion e-commerce application that combines standard shopping functions with an AI-based recommendation module. The recommendation component will use user-provided quantitative attributes (e.g., height and weight as a basic body profile proxy) and qualitative preferences (e.g., Japanese, Korean, or European style categories) to generate more relevant outfit suggestions. By delivering recommendations that are more personalized to the user, the system aims to reduce time spent searching, improve decision confidence, and enhance overall user satisfaction in mobile fashion shopping.

1.3 Advantages and Contributions

This section summarises the expected advantages of the proposed mobile fashion e-commerce application and the contributions it offers to consumers, the fashion retail industry, and academic research. The system targets "mobile-first fashion shoppers" (e.g., young adults and frequent online shoppers) who prefer fast product discovery and need greater confidence in selecting clothing that matches both "fit" and "style".

1.3.1 Advantages for Consumers

The proposed system provides several consumer-focused advantages, enabled by its hybrid personalisation approach (rule-based filtering + ML/content-based ranking):

Improved fit suitability and reduced sizing mismatch - Users enter basic body profile information (e.g., height and weight). The system applies "rule-based filtering" to estimate a suitable size range (e.g., S/M/L group or BMI-based category) and prioritises products with matching size availability. This directly reduces the likelihood of selecting items that do not fit, which can help lower dissatisfaction and returns.

More relevant style matching through personalised ranking - Users select preferred fashion styles (e.g., Japanese, Korean, European). After unsuitable items are filtered out, a lightweight "ML/content-based ranking method" prioritises products whose attributes (e.g., category, style tags, fit type, colour, and material) best match the user's chosen style. This makes recommendations more meaningful than generic "popular items" or purchase-history-only suggestions.

Faster product discovery on mobile devices - By narrowing the catalogue into a focused set of suitable items and ranking them by relevance, users can find appropriate outfits more quickly without extensive scrolling and repeated filtering. This is especially beneficial in mobile shopping contexts where user patience and screen space are limited.

Higher confidence in purchase decisions - Recommendations that consider both body profile and explicit style preference reduce uncertainty during selection. Together with product details and reviews, users can make decisions more confidently, reducing hesitation and improving satisfaction.

Enhanced engagement through supporting features - Supporting features such as wishlists and ratings/reviews help users save items, compare alternatives, and leverage social proof. These interactions can also provide useful feedback signals (e.g., wishlist actions) to refine recommendation ranking over time.

1.3.2 Contributions to the Industry

The project demonstrates how a hybrid personalisation mechanism can be practically applied in mobile fashion retail to improve customer experience and business performance.

Practical AI-driven personalisation aligned with current retail trends"

The system reflects current trends in "mobile commerce" and "data-driven personalisation", showing how structured user inputs (body profile and style preference) can be translated into actionable recommendations without requiring complex technologies such as full 3D body scanning or virtual try-on.

Adoptable approach for both large retailers and SMEs"

Because the hybrid method combines simple rule-based constraints with lightweight ML/content-based ranking, it can be implemented with manageable data requirements (product attributes and tags) and modest computing resources. This improves feasibility for "small and medium fashion businesses" that want personalisation features but cannot invest in expensive AI infrastructure.

Potential reduction in cart abandonment and return-related costs"

Improving fit and style relevance reduces user uncertainty, which can lower drop-off during browsing and checkout. Better suitability at the recommendation stage also reduces incorrect purchases, which may help reduce return rates and associated reverse-logistics costs.

Support for inclusivity and culturally diverse fashion discovery"

By supporting multiple style categories and accommodating different body profiles, the system promotes a more inclusive shopping experience that can serve diverse user groups and fashion tastes.

1.3.3 Contributions to Research

Academically, this project contributes by exploring a more context-appropriate recommendation approach for fashion, where fit and style are core decision factors.

Hybrid recommender design for apparel: constraints + preference-based ranking - Many conventional recommenders rely heavily on collaborative filtering based on user purchase history. This project focuses on a hybrid approach where "rule-based filtering" enforces fit-related constraints (body profile → suitable size group), and "ML/content-based ranking" prioritises items that match explicit style preferences using structured product attributes (e.g., tags, fit type, material, and colour). This combination targets the “cold-start” situation where user history is limited but personalisation is still needed.

Integration of cultural style preference as an explicit recommendation signal - The project treats cultural style preference (e.g., Japanese/Korean/European) as a first-class input rather than an implicit pattern inferred only from behaviour. This supports research into fashion recommendation models that are more interpretable and adaptable to diverse aesthetic preferences.

Foundation for evaluation and future extensions - The work can serve as a baseline for comparing hybrid recommendation performance with alternative approaches discussed in the literature (e.g., pure collaborative filtering, content-based filtering, or deep-learning-based image recommenders). Future research extensions may include richer body measurements, image-based style extraction, and learning-to-rank methods using larger real-world interaction datasets.

1.4 Project Plan

The project will be developed in stages to ensure structured progress and proper evaluation:

- **Phase 1: Core Services (July – September 2025)**
 - Conduct background study and finalize system requirements.
 - Implement essential app features such as product browsing, account management, shopping cart, and simulated checkout.
 - Develop and integrate the AI-powered recommendation engine for personalized outfit suggestions.
- **Phase 2: Extended Services (October – December 2025)**
 - Add wishlist functionality, ratings, and comments.
 - Build user profile management features (edit details, view history).

- Optionally design and implement an admin panel for product management.
- **Phase 3: Testing and System Preview (January – March 2026)**
 - Create detailed test plans and test cases.
 - Conduct functional, usability, and performance testing.
 - Present a prototype for review and refine based on feedback.
- **Phase 4: Finalization and Documentation (March – April 2026)**
 - Complete integration of all features.
 - Finalize report submission and system documentation.

This phased approach ensures that the project is completed on time while allowing room for testing, feedback, and improvements.

1.5 Project Team and Organization

This project is undertaken by:

- **Student Researcher:** Sia Jin Sheng (RSD2S3 G6)
- **Supervisor:** Ms. Siti Nadiah Binti Nain

As this is an individual project, the student researcher bears full responsibility for the “end-to-end planning and execution” of the work. This includes defining the project scope and requirements, selecting appropriate methods (including decisions related to the hybrid recommendation approach), designing the system architecture and database, implementing the mobile application, conducting testing and evaluation, and producing all project documentation and reporting.

The supervisor’s role is to provide academic guidance by reviewing progress, advising on research direction and methodology, ensuring that the project remains feasible and relevant to FYP expectations, and evaluating deliverables at each milestone.

Overall, this structure supports clear accountability and efficient communication throughout the project lifecycle, while emphasising the student's independent role in technical, research, and managerial decision-making.

1.6 Chapter Summary and Evaluation

This chapter introduced the proposed mobile fashion e-commerce application by presenting the project objectives, background, advantages and contributions, project plan, and project organization. The project is motivated by a key limitation in current online fashion shopping: users often face difficulty selecting clothing that matches both "fit requirements" and "style preferences" due to insufficient personalisation in existing retail applications.

From an evaluation perspective, the proposed solution is suitable and feasible for the project scope. The system adopts a "hybrid recommendation approach", where rule-based filtering can ensure practical fit-related constraints (using body profile inputs), while lightweight ML/content-based ranking can improve style relevance without demanding large-scale real-world purchase-history datasets. This makes the implementation achievable within an FYP timeline while still addressing the core problem of personalisation in a measurable and technically justified manner. In addition, the planned development phases and milestones provide a structured pathway to deliver the core e-commerce functions first, followed by recommendation development and user evaluation.

This chapter also establishes the rationale for conducting a literature review in the next chapter. Since recommendation systems and AI personalisation can be implemented through multiple approaches (e.g., collaborative filtering, content-based methods, hybrid models, and fashion-oriented techniques), "Chapter 2" will review relevant studies and existing solutions to (i) justify the selected hybrid approach.

(ii) identify suitable datasets/features and evaluation methods.

(iii) refine the system design based on proven practices and gaps in prior work.

Chapter 2: Literature Review

2.1 Introduction

This chapter presents a literature review of studies, technologies, and existing systems related to the development of a mobile fashion e-commerce application with personalised outfit recommendations. The purpose of this literature review is to (i) "justify the project problem" (limited fit- and style-based personalisation in current online fashion shopping), (ii) "support the proposed hybrid AI solution" by reviewing established recommendation approaches, and (iii) "guide key design decisions" such as the choice of recommendation method, input features (e.g., body profile and style preference), and suitable evaluation strategies.

To achieve this, the chapter is organised as follows. First, it reviews existing work on "fashion recommendation systems" and how personalisation is commonly implemented in apparel contexts. Next, it discusses relevant "AI and machine learning techniques" used in recommender systems, with emphasis on approaches that are practical for limited-data scenarios (e.g., content-based and hybrid methods). Finally, it examines "related commercial fashion applications and e-commerce design practices" to identify common features, limitations, and gaps that the proposed system aims to address. This provides a theoretical and practical foundation for the system design and methodology presented in later chapters.

2.2 Related Studies on Fashion Recommendation Systems

Fashion Recommender Systems (FRS) aim to improve the clothing selection process by suggesting items that match a user's needs and preferences. Compared to general product recommenders, FRS face additional challenges because clothing suitability depends not only on "taste" (e.g., preferred aesthetics) but also on practical constraints such as fit, body shape, and styling compatibility. As a result, research in this area often combines multiple data sources (e.g., user behaviour, item attributes, and visual features) to produce more useful recommendations.

Recent studies commonly report that "hybrid recommenders" perform better for fashion contexts than single-method approaches. Liu et al. (2023) show that combining user-based signals (e.g., similar users' interactions) with content-based features improves recommendation accuracy, because it reduces the weaknesses of relying purely on popularity or collaborative filtering alone. This is especially relevant in fashion platforms where new items are introduced frequently (seasonal collections), causing sparse interaction data and "cold-start" issues. In such situations, content-based information (such as category, brand, fabric, and style tags) provides an important fallback to maintain recommendation quality.

At the same time, other researchers emphasise that fashion recommendations should incorporate "fit-related user information", since clothing is strongly constrained by physical suitability. Park and Yoo (2022) demonstrate that using basic body profile inputs (e.g., height and weight) improves the relevance of outfit suggestions compared to approaches that primarily rely on visual similarity. In comparison to Liu et al. (2023), which focuses on improving predictive accuracy through hybrid filtering methods, Park and Yoo (2022) contribute by improving the *practicality* of recommendations (i.e., items that a user is more likely to wear comfortably). Together, these studies suggest that effective fashion recommendation requires both (i) preference modelling and (ii) fit-aware constraints.

However, a limitation across many existing approaches is that personalisation is often built around either behavioural data (purchase/click history) or visual/content similarity, with less attention given to "explicit style preference definitions", such as culturally influenced styles (e.g., Japanese, Korean, European). While some systems can implicitly learn style patterns through user behaviour, this is less effective for new users and does not always reflect how real shoppers describe their preferences (users often think in terms of style categories). Importantly, relatively few implementations combine "basic physical attributes (fit-related data)" together with "explicit cultural style preferences" as direct inputs to the recommendation process.

This limitation directly motivates the proposed project: the system is designed to address the gap by implementing a "hybrid recommendation approach" that (1) applies rule-based filtering using body profile inputs to ensure size suitability, and (2) applies lightweight ML/content-based ranking using item attributes and style tags to prioritise products consistent with the user's selected cultural style preference. By doing so, the proposed system aims to provide recommendations that are both "wearable (fit-aware)" and "aesthetically aligned (style-aware)", even when detailed purchase history is unavailable.

To strengthen domain coverage, additional studies on fashion recommendation also commonly highlight the importance of real-world constraints such as sparsity and explainability. In many deployed e-commerce environments, the availability of high-quality user-item interaction data may be limited, particularly for new platforms or smaller retailers. Therefore, approaches that rely on structured item metadata and explicit user inputs remain practical and relevant for mobile fashion applications, supporting the feasibility of the proposed hybrid design in this project.

2.3 Artificial Intelligence and Machine Learning in Fashion

Artificial Intelligence (AI) and Machine Learning (ML) are widely used in fashion retail to support personalisation, product discovery, and customer decision-making. In large-scale fashion platforms, deep learning techniques are often applied to unstructured data. For example, Convolutional Neural Networks (CNNs) are commonly used for clothing image recognition (e.g., detecting categories, colours, patterns, and visual style cues), while Natural Language Processing (NLP) techniques can support matching and analysing text such as product descriptions, reviews, and search queries. These approaches are discussed to show how advanced fashion recommendation systems are implemented in industry, even though they are not fully implemented in this project due to scope constraints.

For this project, the recommendation component adopts a hybrid approach that is more feasible within an FYP timeline and limited dataset conditions. The system first applies rule-based filtering using user body profile inputs (e.g., height and weight) to ensure basic fit suitability, then applies a lightweight ML/content-based ranking method to prioritise items that match the user's selected style preference (e.g., Japanese/Korean/European). A pre-trained or pre-defined lightweight model is preferred because it reduces development and training time, avoids the need for large-scale labelled datasets and lengthy model tuning, and can run efficiently on typical student computing resources.

Table 2.X compares common recommendation-related algorithms used in fashion applications and explains why the hybrid method is the most suitable choice for this project. In particular, collaborative filtering is less appropriate because it depends

heavily on user-item interaction history, which is usually unavailable for new systems (cold-start). Content-based filtering is more practical because it can use product attributes such as style tags and fit type. Therefore, the hybrid approach offers the best balance by combining fit-aware filtering with style-aware ranking.

Algorithm	Application in Fashion	Relevance to Project
Collaborative Filtering	Recommends items based on similar users' interactions (clicks, purchases, ratings)	Low—does not use personal dataLow — relies on large user-item history and does not directly use body profile or explicit style preference
Content-Based Filtering	Matches items using product attributes/tags (e.g., category, colour, style tags, fit type, material)	Medium — suitable for style-driven recommendations using item metadata
Hybrid Model	Combines multiple signals (e.g., rule-based constraints + content-based/ML ranking, or CF + content-based)	High — used in this project to enforce fit suitability and improve style relevance
Deep Neural Networks	Learns complex relationships using images/text features (CNNs, transformers)	Medium — relevant conceptually, but full implementation is not feasible due to time, data, and resource constraints

2.4 E-Commerce Application Features and Design

E-commerce applications such as H&M, Uniqlo, and Zara typically provide essential mobile shopping features including product browsing by category, product detail pages, wishlists, cart management, checkout flow, and user authentication. These features form the baseline user expectation for any fashion shopping application. However, their recommendation functions are often limited to non-personalised methods such as trending items, “new arrivals”, or simple category-based suggestions. This limitation supports the need for a system design that prioritises stronger personalisation while still maintaining standard e-commerce usability.

The design of the proposed system is therefore influenced by two key considerations: (1) meeting core e-commerce functionality requirements, and (2) improving user decision-making through personalisation. In practice, this means the application must implement a familiar shopping flow (browse → view details → save to wishlist/cart → checkout) while integrating AI recommendations in a way that does not complicate navigation. To support this, the interface should emphasise simplicity and clarity (e.g., minimal steps to reach product details, consistent layout for categories, and clear call-to-action buttons). This is important because mobile users tend to abandon sessions when browsing becomes time-consuming or confusing.

AI-driven personalisation is included not only as a technical enhancement but also as a user engagement strategy. Chiu and Wong (2021) reported that AI-driven personalisation can increase customer engagement by 35% in e-commerce environments. In this project, this motivates the inclusion of: (i) a personalised recommendation section that suggests fit- and style-relevant outfits, and (ii) feedback-related features such as ratings, reviews, and wishlists. These features support engagement by giving users more reasons to interact with the app (saving items, evaluating products, and comparing options), and they also provide signals that can be used to improve recommendation relevance over time.

In addition, trust and usability are critical when introducing AI-based recommendations in fashion shopping. Since users may not immediately trust automated suggestions, the system should present recommendations transparently and provide supporting information (e.g., “recommended based on your selected style” or showing matched tags such as fit type and style category). Ratings and reviews further strengthen trust by allowing users to validate product quality and fit feedback from other customers. Together, these elements reduce uncertainty and help users feel more confident when relying on recommendations.

Key e-commerce design considerations reflected in this project include:

- **Usability (mobile-first simplicity):** clean layout, fast navigation, and minimal steps from browsing to checkout to reduce friction in mobile shopping.
- **Security and privacy:** secure authentication and careful handling of personal profile inputs (e.g., body profile data), as these affect user willingness to adopt the system.
- **Responsiveness and performance:** consistent behaviour across different screen sizes and efficient loading of product lists and recommendations to avoid delays that may lead to cart abandonment.

Overall, this section supports the project’s design direction: implement standard fashion e-commerce features to meet user expectations, then enhance the shopping experience using AI-driven personalisation and trust-building feedback mechanisms.

2.5 Related Systems Comparison

System	Features	AI Recommendation	Personalization	Limitation
H&M App	Product browsing, cart, wishlist, account	No	Mostly manual filters (category/size/color), trending items	Does not use user body profile (e.g., height/weight) and does not provide style-based outfit recommendations; users still need to manually judge fit and suitability
Uniqlo App	Product catalogue, membership/loyalty, store locator, promotions	No (primarily rule/promotional driven)	Limited (often campaign-based; may reflect	Recommendations are not designed as AI outfit suggestions and do not explicitly combine fit needs with

			purchase/member activity)	style preferences for personalised discovery
Zalora	Product browsing, cart, promotions, brand-focused discovery	Basic (commonly collaborative-filtering style “similar users/items”)	Moderate (based on browsing/purchase behaviour)	Personalisation depends mainly on behaviour/history; weak for new users (cold start) and does not explicitly incorporate body profile or cultural style preference inputs
Proposed System	Browsing, cart, wishlist, ratings/reviews, user profile	Yes (hybrid approach)	High (fit-aware + style-aware)	To be validated during development and user evaluation (implementation and dataset scope are project constraints)

Analysis and justification

Existing fashion retail apps succeed at providing standard e-commerce functions, but they often fall short in addressing the main user need in online apparel shopping: choosing items that match both fit and style without trying them on. In systems such as H&M and Uniqlo, personalisation is generally limited to browsing filters (e.g., size, category, colour) and promotional or popularity-driven listings. These tools help users narrow the catalogue, but they do not actively guide users toward outfits that are suitable for their body profile and preferred styling. As a result, the user must still make most decisions manually, which increases time spent browsing and increases uncertainty about suitability.

Platform-style retailers such as Zalora may provide “recommended” items, but these are commonly driven by behaviour-based methods (e.g., collaborative filtering using clicks or purchase history). This type of personalisation has two key weaknesses for fashion contexts:

1. it performs poorly for new users with limited history (cold-start problem), and
2. it does not directly model fit constraints or explicit style intent (e.g., “I want Korean minimalist style that suits my body profile”).

Therefore, even when recommendations exist, they may be relevant in a general sense but not practically helpful for fit and style decision-making.

The proposed system addresses these gaps by introducing explicit inputs that current apps usually do not use together: body profile (height and weight) and style preference (e.g., Japanese/Korean/European). The system applies a hybrid recommendation approach: rule-based filtering first removes items unlikely to be suitable (fit/size constraint), then a lightweight ML/content-based ranking method prioritises products whose attributes and style tags align with the user’s selected style. This directly targets the missing personalisation dimension in existing systems: recommendations that are both fit-aware and style-aware, rather than only popularity-based or behaviour-based.

Importantly, the proposed system is designed to be realistic and feasible within an FYP scope. Instead of requiring complex technologies such as 3D body scanning, virtual try-on, or training deep neural networks on large image datasets, it uses structured product metadata and explicit user inputs that can be implemented with modest computational resources. This

makes the approach suitable for prototyping within the project timeline, while still demonstrating a meaningful improvement over existing mobile fashion shopping applications.

2.6 Summary and Evaluation

This chapter reviewed literature on fashion recommender systems, AI/ML techniques used in fashion e-commerce, key e-commerce application design considerations, and comparisons with existing market systems. Overall, the reviewed studies confirm that personalisation can improve the usefulness of fashion shopping platforms, but they also highlight practical challenges such as limited user history (cold-start), rapidly changing product inventories, and the need for recommendations to be wearable (fit-aware) rather than only preference-based.

A clear research and technical gap identified from the review is that many existing systems either (i) rely mainly on behavioural data (e.g., collaborative filtering using purchase/click history), or (ii) focus on visual similarity and product metadata, while few solutions explicitly integrate basic body profile information (fit constraints) together with cultural style preferences as direct recommendation inputs. This gap is important because it reflects how users make apparel decisions in real life: shoppers evaluate both fit suitability and style identity before purchasing.

The findings from this chapter directly influenced the system design decisions in this project. First, due to the likely absence of large-scale user interaction datasets, the project adopts a hybrid recommendation model that combines rule-based filtering (to enforce fit suitability using height and weight) with lightweight ML/content-based ranking (to prioritise items aligned with the user's selected style preference using product attributes and tags). This approach is technically feasible within an FYP scope because it reduces the dependency on large training data and heavy computation while still providing meaningful personalisation.

This literature review therefore provides both the justification and the theoretical grounding for the proposed solution. The next chapter will use these insights to define the project methodology and system design, including the system architecture, database structure, recommendation workflow, and evaluation plan.

2.7 Reference

- Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2022). *Recommender systems handbook* (3rd ed.). Springer.
- Aggarwal, C. C. (2016). *Recommender systems: The textbook*. Springer.
- Schafer, J. B., Konstan, J. A., & Riedl, J. (2001). E-commerce recommendation applications. *Data Mining and Knowledge Discovery*, 5(1–2), 115–153.
- Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370.

- Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6), 734–749.
- Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep learning based recommender system: A survey and new perspectives. *ACM Computing Surveys*, 52(1), Article 5.
- McAuley, J., Targett, C., Shi, Q., & Van Den Hengel, A. (2015). Image-based recommendations on styles and substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 43–52). ACM.
- Kang, W.-C., & McAuley, J. (2018). Self-attentive sequential recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)* (pp. 197–206). IEEE.
- Anwar, T., & Umair, M. (2021). A systematic literature review of fashion recommender systems: Techniques, evaluation, and challenges. [Add correct journal/conference details from your source].
- Hassanein, K., & Head, M. (2007). Manipulating perceived social presence through the web interface and its impact on attitude towards online shopping. *International Journal of Human-Computer Studies*, 65(8), 689–708.
- Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.
- ISO. (2019). *ISO 9241-210:2019 Ergonomics of human-system interaction—Part 210: Human-centred design for interactive systems*. International Organization for Standardization.

Chapter 3: Methodology And Requirements Analysis

3.1 Introduction

This chapter outlines the methodology used to develop the proposed mobile clothing e-commerce application with hybrid AI-based outfit recommendations. It explains the requirements gathering and analysis process, including the use of requirement models and diagrams, and defines the functional and non-functional requirements that guide implementation and testing. The methodology presented in this chapter is suitable for this project because it supports structured development of two key components: (i) core e-commerce functions (e.g., browsing, cart, authentication) and (ii) a hybrid recommendation module that must be implemented and evaluated within an FYP timeline, making feasibility and clear planning essential.

In addition, this chapter builds directly on the findings in Chapter 2. The design and requirements are derived from the gaps identified in the literature review—particularly the limited integration of body profile and explicit style preference in existing fashion apps—and from the project’s decision to adopt a practical hybrid recommendation approach.

3.2 Methodology

3.2.1 Selected Development Model: Agile Development

The Agile development model is selected because this project requires iterative refinement of both (i) the mobile e-commerce functions and (ii) the hybrid recommendation feature, where changes are expected after testing and feedback. Instead of completing the system in one pass, Agile allows the project to be developed in small increments (sprints) so that the user interface, recommendation logic, and overall usability can be improved progressively based on supervisor review and limited user testing. This approach is suitable for an FYP because it reduces risk (features are delivered step-by-step), ensures continuous validation, and keeps development aligned with the project objectives and timeline.

In this project, Agile will be applied using **4 planned sprints**, where each sprint produces a working increment of the application. Each sprint includes planning, design/update, development, and testing, followed by review and improvements.

Sprint plan (how Agile is applied in this project):

- **Sprint 1: Requirements and system design (foundation sprint)**
Activities: requirement gathering, functional/non-functional requirement definition, use case diagram and flow design, database schema design, UI wireframes/prototypes.
Output: complete requirement specification and system design documents, validated by supervisor feedback.
- **Sprint 2: Core e-commerce implementation (MVP)**
Activities: develop authentication module, product browsing and product details, category filtering/search (if in

scope), cart and wishlist functions, basic navigation and UI integration.

Output: a working mobile shopping prototype (minimum viable product) that passes functional test cases.

- **Sprint 3: Hybrid recommendation development**

Activities: implement user profile inputs (height, weight, style preference), rule-based filtering for fit suitability, and lightweight ML/content-based ranking using product attributes/tags. Integrate recommendation results into the UI (e.g., “Recommended for You” section).

Output: recommendation module integrated into the app and validated using test scenarios.

- **Sprint 4: Evaluation, refinement, and finalisation**

Activities: usability testing/user evaluation, recommendation relevance evaluation (questionnaire or acceptance test), bug fixing, UI/UX improvement, performance checks, and final documentation.

Output: final tested prototype and evaluation results for reporting and presentation.

Distinction between Project 1 and Project 2 (FYP structure)

To match the FYP workflow, the Agile activities are distributed across two stages:

- **Project 1 (Analysis & Design Focus):**

Main work: Sprint 1 (requirements, diagrams, design) and preparation for implementation (finalised scope, dataset/attributes, UI prototype).

Deliverables: requirements analysis, system design, database design, and development plan.

- **Project 2 (Implementation & Evaluation Focus):**

Main work: Sprints 2–4 (implementation of core modules + hybrid recommender + testing/evaluation).

Deliverables: working mobile application prototype, integrated recommendation module, test results, user evaluation findings, and final report.

This Agile plan ensures the project remains feasible and manageable while giving clear checkpoints for development progress and continuous improvement.

3.2.2 Advantages of Agile for This Project

Copilot said: ### 3.2.2 Advantages of Agile for This

3.2.2 Advantages of Agile for This Project (revised, project-specific)

Agile is beneficial for this project because the system includes two parts that typically require repeated refinement: (i) a mobile e-commerce user experience and (ii) a hybrid recommendation component that depends on profile inputs and product attribute data. The advantages of Agile in this project are as follows:

1. **Supports iterative refinement of the hybrid recommendation module (AI component)**

The recommendation feature will likely require adjustment after early testing (e.g., tuning rule thresholds for size grouping, adjusting feature weights for style matching, or improving product tagging). Agile allows the

recommendation logic to be implemented in an initial simple form and then refined across later sprints based on observed outputs and feedback, rather than being fixed too early.

2. **Enables gradual UI/UX improvement based on real user flow testing**

Mobile commerce usability depends heavily on navigation clarity and screen layout. With Agile, UI screens such as product browsing, product details, cart, and the “Recommended for You” section can be reviewed and improved sprint-by-sprint. This is important because even a correct recommendation algorithm will not be effective if users cannot easily understand or access the recommended items.

3. **Manages uncertainty in data availability and feature scope**

The recommendation system depends on the availability and quality of product attributes (e.g., style tags, fit type, sizes). Agile reduces risk by allowing the project to start with a minimum viable dataset and baseline metadata, then expand or adjust the dataset/features in later sprints if constraints arise. This prevents project failure due to over-committing to a complex model or unrealistic data collection requirements early on.

4. **Reduces overall project risk through incremental deliverables**

Each sprint produces a working increment (e.g., a functioning browsing and cart system, then integrated recommendations, then evaluation and refinement). This reduces the risk of reaching late stages with an incomplete system, and it ensures that core requirements are completed before optional enhancements.

5. **Improves decision-making through continuous supervisor feedback**

Regular sprint reviews make it easier to confirm that the project remains aligned with FYP expectations, especially for the AI scope (hybrid approach). Supervisor feedback can be applied early—for example, to adjust the recommendation method, evaluation approach, or feature prioritisation—before major implementation effort is invested.

Overall, Agile helps this project remain feasible by supporting controlled iterations, reducing uncertainty around the AI component, and ensuring continuous improvement of both recommendation quality and mobile user experience.

3.3 Requirements Gathering Techniques

The system requirements were gathered using both primary and secondary methods. These techniques are sufficient for an FYP because they combine (i) benchmarking real apps, (ii) input from potential users, and (iii) academic guidance to keep the scope feasible.

How each method influenced requirements:

- **Observation (H&M, Uniqlo, Zara, Zalora):** identified baseline requirements such as category browsing, product details (price/size/images), cart, wishlist, and familiar mobile UI navigation.
- **Online survey:** (state whether conducted or planned) used to identify requirements for user profile inputs (height, weight, style preference), user acceptance of AI recommendations, and privacy expectations (minimal data collection, ability to edit profile).
- **Literature review:** justified choosing a hybrid recommender (rule-based fit filtering + lightweight ML/content-based ranking) and guided what product attributes/tags are needed.

- **Supervisor consultation:** confirmed scope and prioritisation (core modules first), and guided realistic testing/evaluation for the FYP timeline.

Method	Description	Expected Outcome
Observation	Review of existing fashion e-commerce apps (H&M, Uniqlo, Zara, Zalora)	Defines baseline features and UI standards (browse, product detail, cart, wishlist, navigation patterns)
Online Surveys	Gather user preferences on AI outfit recommendations and privacy concerns	Defines user profile inputs, privacy needs, and how recommendations should be presented; informs acceptance criteria
Literature Review	Analyze existing research on recommender systems and e-commerce design	Justifies hybrid recommendation choice; identifies required item attributes and evaluation approach
Consultation with Supervisor	Discuss scope, limitations, and risk management	Confirms feasibility, sets priorities, and finalises scope for FYP deliverables

3.4 Requirements Analysis

The requirements analysis focuses on defining system components, data flow, and interactions between users and the system.

3.4.1 Use Case Diagram

Actors (with justification)

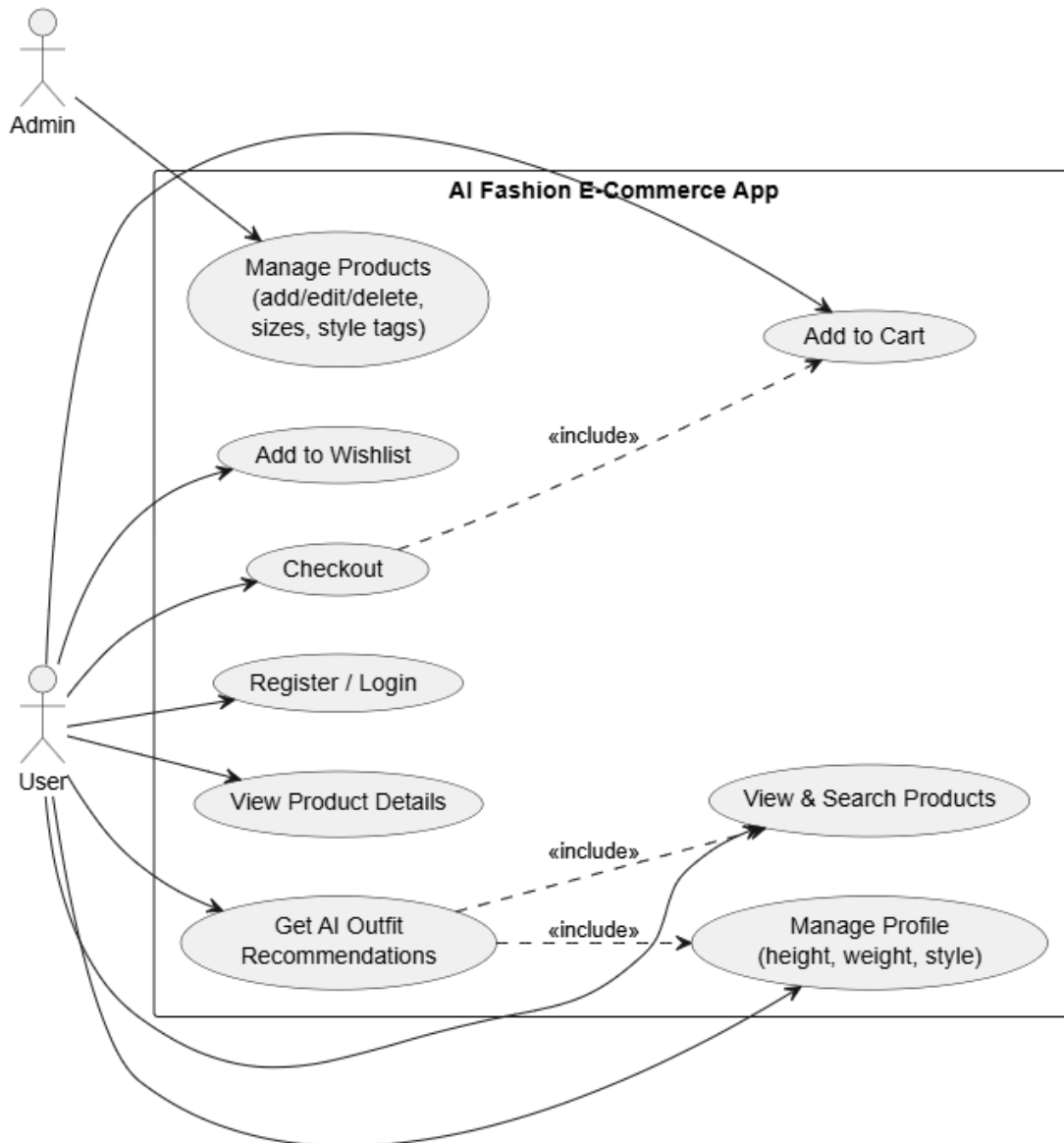
- **User:** primary actor who performs all shopping and personalisation actions (browsing, cart, recommendations, profile, ratings).
- **Admin:** included to support minimal back-office control so the app is realistic (e.g., adding/updating products, prices, sizes, and style tags). Without an Admin function, the product catalogue and recommendation inputs cannot be maintained properly.

Note: Admin scope is limited to product/catalogue management for this project.

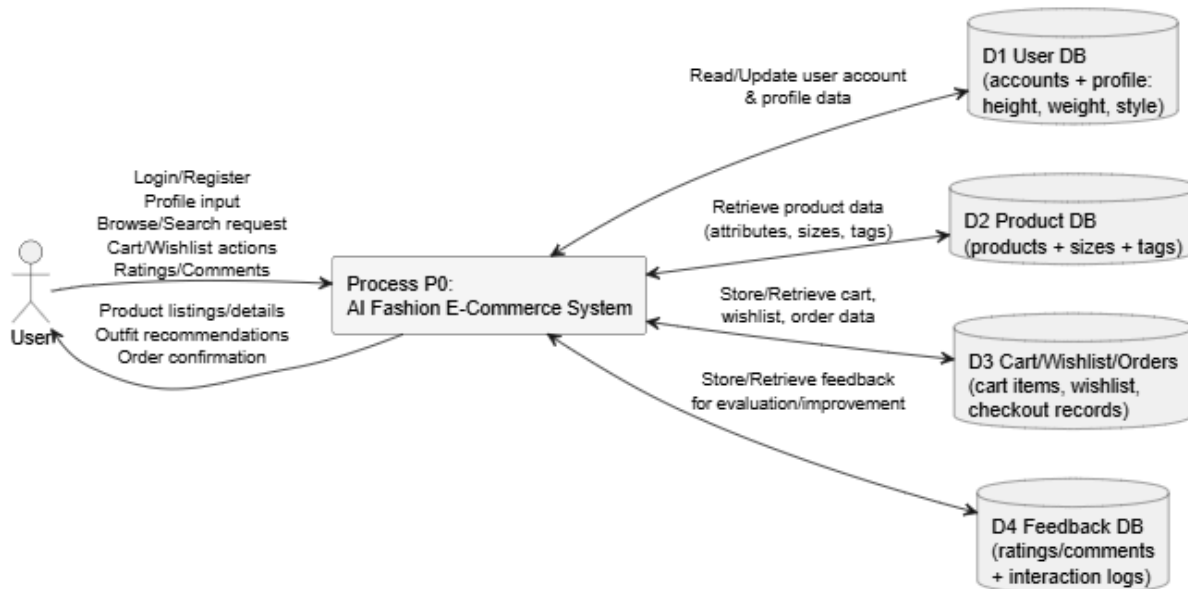
Use Cases (mark scope) In-scope for Project 2 (implementation):

- User registration/login
- View/search products
- Add to cart/wishlist

- Checkout (basic flow)
- Manage user profile (height, weight, style preference)
- AI outfit recommendation (hybrid)
- Admin: product management (add/edit/delete product, sizes, style tags)



3.4.2 Data Flow Diagram (DFD) – Level 0



Explanation (Level 0 DFD):

- **External entity (User):** provides inputs such as login/registration details, profile data (height, weight, style preference), browsing/search requests, cart/wishlist actions, and ratings/comments.
- **Process P0 (System):** receives user inputs, retrieves product data, generates recommendations, and manages shopping actions (cart/wishlist/checkout).
- **Data stores:**
 - D1 User DB: stores account credentials and user profile (height, weight, style preference).
 - D2 Product DB: stores product catalogue data including sizes and style tags used for recommendations.
 - D3 Cart/Wishlist/Orders: stores shopping state and checkout records.
 - D4 Feedback DB: stores ratings/comments (and optionally interaction logs) to support later evaluation and iterative improvement.
- **Outputs back to the user:** product listings/details, recommended items, and checkout/order confirmation.

Why Level 0 is sufficient now: it shows the full system boundary and the main data movement between the user, the core system, and databases. If required later, a Level 1 DFD can split P0 into sub-processes (Authentication, Product Browsing, Recommendation Engine, Cart/Wishlist, Checkout, Feedback).

3.5 Functional and Non-Functional Requirements

3.5.1 Functional Requirements

ID	Functional Requirement
FR1	The system shall allow a user to register an account using email/username and password.

FR2	The system shall allow a user to log in and log out using valid credentials.
FR3	The system shall allow a user to view the product catalogue and filter products by gender and category.
FR4	The system shall allow a user to filter or sort products by style tag (e.g., Japanese/Korean/European).
FR5	The system shall allow a user to view product details (name, price, images, available sizes, style tags).
FR6	The system shall allow a user to create/update their profile (height, weight, style preference).
FR7	The system shall generate recommended products/outfits based on the user's profile and product attributes, and display them in a recommendation list.
FR8	The system shall allow a user to add items to cart, update quantity, and remove items from cart.
FR9	The system shall allow a user to add/remove items in the wishlist and view the wishlist list.
FR10	The system shall allow a user to place an order by confirming checkout (payment can be simulated if the real gateway is out of scope).
FR11	The system shall allow an admin to add, edit, and delete product listings, including sizes and style tags.
FR12	The system shall allow an admin to view a list of user orders, including order ID, user ID/name, order date/time, total amount, and order status.

3.5.2 Non-Functional Requirements

ID	Non-Functional Requirement	Justification (brief)
NFR1	Performance: The system shall display recommendations within 3 seconds after the user requests them.	Mobile users expect fast responses; 3 seconds is a practical target for a prototype running lightweight filtering/ranking without high-end servers.

NFR2	Security: The system shall protect authentication and personal data by using HTTPS and encrypted password storage (hashing + salt).	The app stores sensitive profile data (height/weight) and login credentials, so basic protection is required even for an FYP prototype.
NFR3	Usability: The UI shall be mobile-friendly and responsive, and core tasks (browse → cart → checkout) shall be achievable with minimal steps.	The system is a mobile app; usability directly affects whether users will use recommendations and complete purchases.
NFR4	Scalability: The system shall support adding more products and style tags without major code changes (e.g., database-driven catalogue).	Realistic for FYP: user base is small, but the product catalogue may grow during testing; design should not assume a fixed dataset.
NFR5	Maintainability: The code shall be modular (separate UI, backend/API, database, recommendation module).	Makes it easier to debug and enhance features (e.g., changing the recommender) within limited FYP time.
NFR6	Reliability: During evaluation/testing, the prototype shall run without crashes for $\geq 95\%$ of test sessions.	For an FYP prototype, “uptime” is best measured during test sessions rather than real production deployment; 95% is a reasonable stability target.

3.6 Chapter Summary and Evaluation

This chapter defined the methodology and requirements for developing the proposed mobile fashion e-commerce application with hybrid AI recommendations. Agile was selected because it supports incremental delivery of core modules (authentication, browsing, cart, checkout) while allowing the recommendation logic and UI to be refined through sprint reviews and testing. Defining clear functional and non-functional requirements increases project feasibility by setting measurable implementation targets and preventing scope creep within the FYP timeline.

A key project risk is limited data availability for training and validating recommendation quality, as well as potential integration issues between the recommendation module and the mobile UI/database. These risks are managed through the Agile approach by implementing a minimum viable version early (basic catalogue + profile + recommendations), then improving it iteratively based on test results and supervisor feedback.

The outputs of this chapter (actors, DFD, use cases, and requirement specifications) directly guide Chapter 4 by translating the requirements into system architecture, database design, module design, and detailed implementation planning.

Chapter 4: System Design

4.1 System Overview / Workflow Design

This section provides a high-level overview of how the clothing e-commerce application with AI outfit recommendation operates from input to output.

4.1.1 System Workflow Steps

The system workflow is designed to support a standard mobile e-commerce flow (browse → product → cart → checkout) while integrating AI recommendations as an optional entry point. AI recommendations are optional because users may prefer manual browsing, and recommendations require profile inputs (height, weight, style). The recommendation results are integrated into the same product detail and checkout flow to keep the user experience consistent.

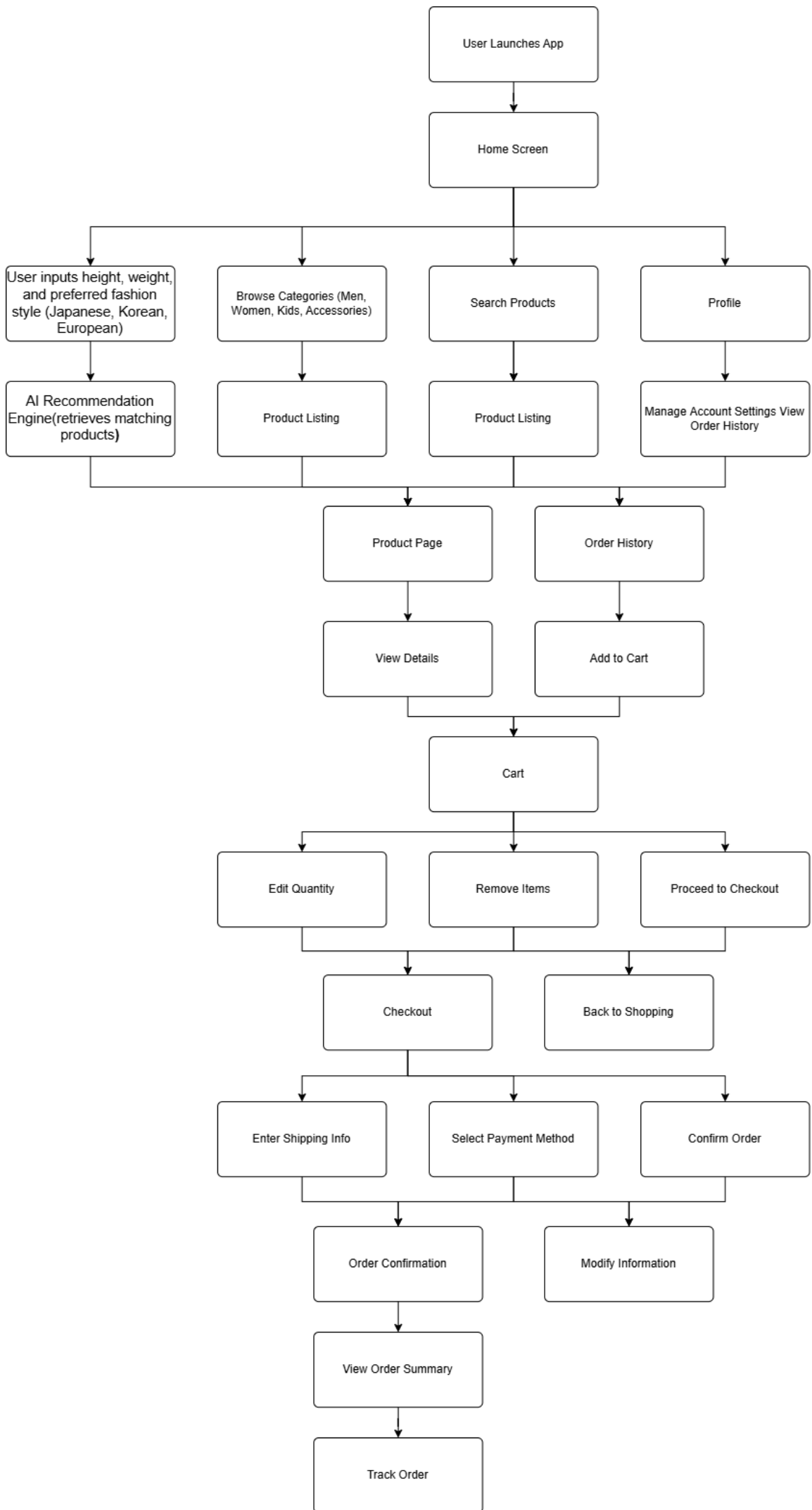
Main workflow (key interactions and data actions):

1. Access products (two entry paths)
 - Path A: Browse/Search (default): user browses by category or searches keywords.
 - Data read: Product Database (listings, prices, sizes, style tags).
 - Path B: AI Recommendations (optional): user enters/updates profile (height, weight, style preference) and requests recommendations.
 - Data write/read: User Database (store profile) + Product Database (retrieve candidate items) → Recommendation Engine returns ranked items.
2. View product details
 - The user selects an item from browse/search/recommendations.
 - Data read: Product Database (details, images, sizes, tags).
3. Save intent (wishlist/cart)

- The user adds an item to the wishlist or cart.
 - Data write: Wishlist/Cart Store (save or update items/quantity).
- 4. Checkout and order creation
- The user enters shipping details and confirms checkout (payment can be real or simulated depending on scope).
 - Data write: Orders Store (order record, items, totals, status).
- 5. Post-purchase management
- User views order history and basic tracking/status.
 - Data read: Orders Store (past orders and current status).
- Users can update their profile at any time.
 - Data write: User Database.

Why the order is designed this way

- It keeps the core experience aligned with familiar e-commerce behaviour, reducing user confusion.
- Recommendations are placed as an optional path so the system still works fully without AI, and users can choose between manual discovery and guided discovery.
- All paths converge at the same product and checkout screens, which simplifies implementation and avoids duplicated UI flows.



4.1.2 Workflow Diagram (Description)

The workflow diagram visualises the complete end-to-end user journey and confirms that the system supports a consistent shopping process while integrating AI recommendations as an optional path. The Home Screen functions as the navigation hub, branching into two discovery methods: manual browsing/search and AI-assisted recommendations.

A key purpose of the diagram is validation of system completeness and flow consistency. It shows that regardless of how products are discovered (category/search or AI recommendation), both paths converge at the same Product Page, then follow the same downstream process: Wishlist/Cart → Checkout → Order Confirmation → Order Summary/Tracking. This proves the design avoids separate “AI-only” purchasing flows, reducing complexity and ensuring users do not face inconsistent interfaces or duplicated steps.

The diagram also highlights the main decision point where AI is triggered: the user chooses the AI recommendation feature and provides profile inputs (height, weight, style). This confirms that AI is treated as a supporting feature rather than a requirement, meaning the app remains usable even if the user skips recommendations or has incomplete profile data.

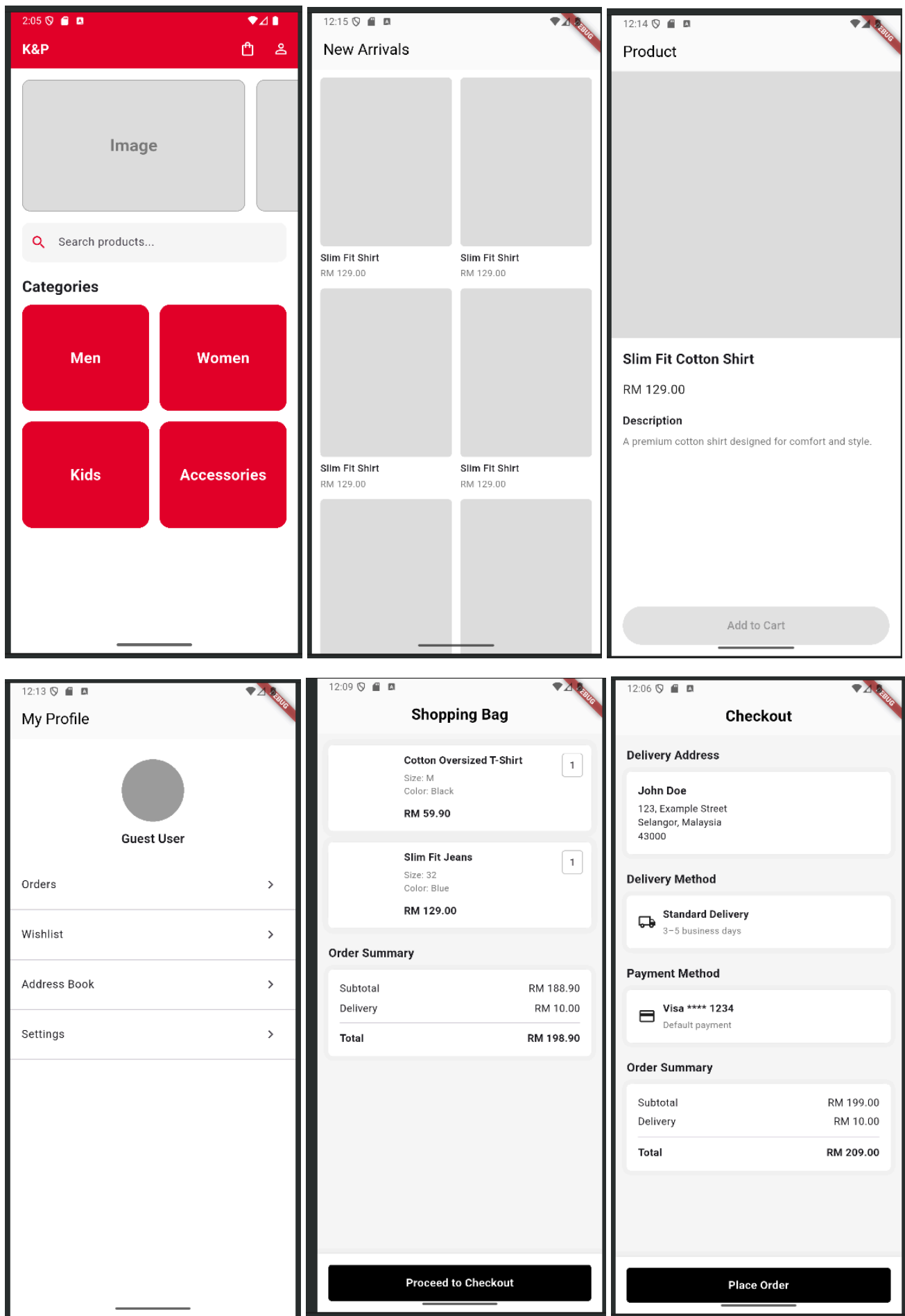
Assumptions / limitations at this stage

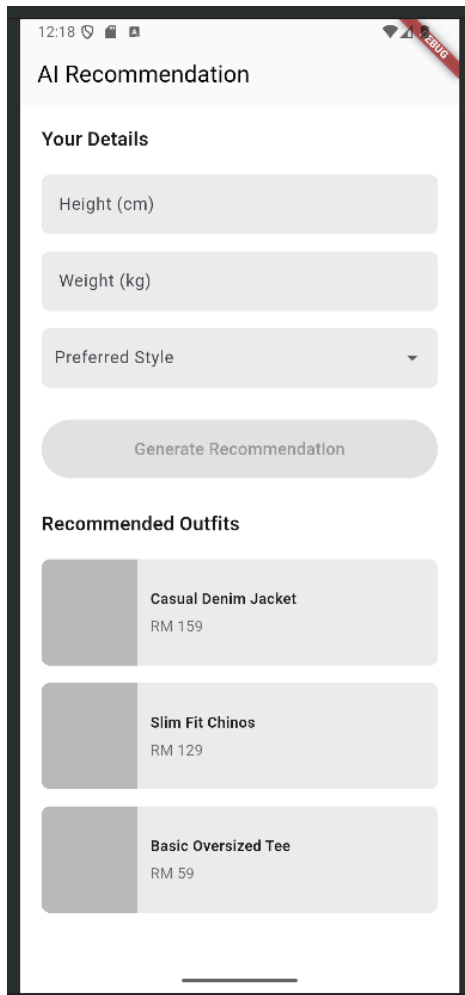
- Payment may be implemented as a simulated confirmation (no real payment gateway) depending on scope and time constraints.
- Order tracking may be limited to basic status updates (e.g., “Processing/Delivered”) rather than real courier integration.
- Recommendation quality depends on available product attributes/tags; the diagram assumes products have usable sizes and style tags for filtering/ranking.

Overall, the workflow diagram demonstrates that the proposed system is well-structured: it covers the full purchase lifecycle and integrates AI recommendations without disrupting standard e-commerce navigation.

4.2 User Interface (UI) / System Interaction Design

4.2.1 UI Wireframe Description





The wireframes presented consist of **seven low-fidelity demo screens** used to communicate the initial UI structure and user flow. They are **not final** and will be refined iteratively during **Project 2** based on implementation constraints and usability feedback. The design prioritises a **mobile-first layout**, clear navigation, and minimal steps to complete key tasks (discover → view → save → purchase).

Home Screen

Design decision: key entry points (Categories, “Recommended for You”, Trending) are placed on the Home Screen to reduce user effort and improve navigation efficiency.

Usability rationale: showing categories first supports quick scanning and lowers cognitive load for new users, while “Recommended for You” provides a fast path for returning users who want personalised discovery.

Login & Registration Screen

Design decision: kept minimal (email + password) to reduce friction during onboarding.

Usability rationale: fewer fields improve completion rate on mobile; integration with Firebase/Flutter authentication is planned to support secure login without complex custom development.

Product List Screen

Design decision: products are displayed in a simple card layout with essential actions (View Details / Add to Cart).

Usability rationale: fewer fields improve completion rate on mobile; integration with Supabase Auth in Flutter is planned to support secure login without complex custom development.

Product Details Screen

Design decision: large image and key info (price, description, size) are prioritised; action buttons (Add to Cart / Wishlist) are prominent.

Usability rationale: users decide visually in fashion apps, so images are emphasised; primary CTAs are made easy to reach to support faster decision-making.

Note: ratings/comments are shown only if implemented; otherwise this becomes a future enhancement section.

AI Recommendation Screen

Design decision: recommendation is accessed as a dedicated screen with simple inputs (height, weight, style) and a single “Generate” action.

Usability rationale: structured input reduces confusion and ensures the system receives the minimum data needed; placing AI access clearly supports the project’s key differentiator without interrupting manual browsing.

Cart & Checkout Screen

Design decision: cart review and checkout are combined to keep the purchase flow short.

Usability rationale: fewer steps reduces drop-off on mobile. A simulated payment option is included to keep the prototype feasible while still demonstrating the full order workflow.

User Profile Screen

Design decision: profile editing and order history are grouped in one place.

Usability rationale: centralising account-related actions reduces navigation complexity and supports personalisation (profile updates directly affect recommendation output).

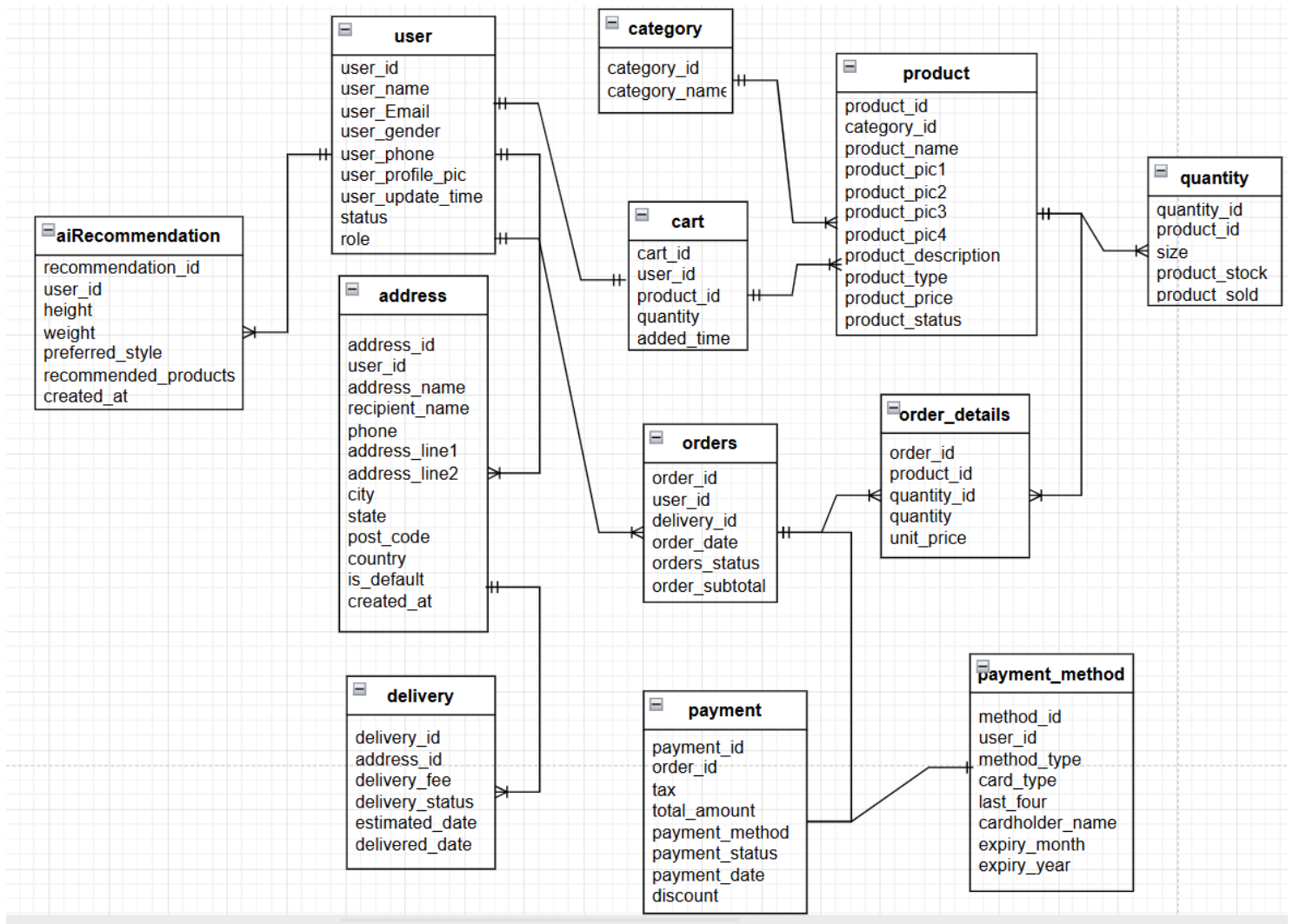
4.3 Data Design

4.3.1 Entity Relationship Diagram (ERD)

Your ERD includes these entities:

Entity	Key Attributes	Justification / Purpose
User	user_id , user_name, user_email, user_gender, user_phone, user_profile_pic, user_update_time, status, role	Stores user identity for login, profile management, and linking to cart/orders. role/status supports Admin vs User access control.
Category	category_id , category_name	Supports structured browsing/filtering of products (FR3).
Product	product_id , category_id, product_name, product_pic1, product_pic2, product_pic3,	Core catalogue data displayed in product list/details; linked to Category and inventory sizes.

	product_pic4, product_description, product_type, product_price, product_status	
Quantity	quantity_id , product_id, size, product_stock, product_sold	Separated to manage multiple sizes per product with stock tracking, which is essential for clothing inventory.
Cart	cart_id , user_id, product_id, quantity, added_time	Stores each user's temporary shopping items and quantity (FR8).
Orders	order_id , user_id, delivery_id, order_date, orders_status, order_subtotal	Stores confirmed purchases, supports order history and admin "view orders" activity (FR12).
Order_Details	order_id , product_id, quantity_id, quantity, unit_price	Separated from Orders to support one order → many items and to keep purchase-time values (quantity/price) consistent even if Product price changes later.
Address	address_id , user_id, address_name, recipient_name, phone, address_line1, address_line2, city, state, post_code, country, is_default, created_at	Stores shipping addresses for checkout and allows users to manage default address (FR10).
Delivery	delivery_id , address_id, delivery_fee, delivery_status, estimated_date, delivered_date	Supports basic delivery status after order. Prototype note: tracking is limited to status fields (no real courier API integration).
Payment	payment_id , order_id, user_id, tax, total_amount, payment_method, payment_status, payment_date, discount	Records checkout confirmation and totals. Prototype note: payment is simulated (no real payment gateway), but stored to show complete purchase flow.
Payment_Method	method_id , user_id, method_type, card_type, last_four, cardholder_name, expiry_month, expiry_year	Stores saved payment method metadata for convenience. Prototype note: can be minimal or optional if payment is simulated.
ai_Recommendation	recommendation_id , user_id, height, weight, preferred_style, recommended_products, created_at	Stored separately to record user inputs and generated results for the recommendation feature (FR7), and to support evaluation/debugging without mixing with core order data.



4.3.2 Class Diagram

Class	Module Mapping (UI / Backend / AI)	Key Attributes (match ERD)	Major Methods (what it does)	Notes on Consistency
User	UI + Backend	user_id, user_name, user_email, height, weight, style_preference, gender, phone, profile_pic	login() verify credentials; updateProfile() update height/weight/style; viewRecommendations() triggers recommendation flow for user	Matches User table; profile fields align with ERD user/ai inputs.
Product	UI + Backend	product_id, product_name, category_id, description, product_type, price, status, images[], stock	getDetails() return full product details; getStock() retrieve size availability	images[] groups product_pic1-4; stock can be derived (actual per-size stock in Quantity).

Category	UI + Backend	category_id, category_name	listProducts() returns products under selected category	Direct mapping to Category table.
Quantity (ProductVariant)	Backend	quantity_id, product_id, size, stock, sold	updateStock() update stock after order/cancel	Direct mapping to Quantity table (size-level inventory).
Cart	UI + Backend	cart_id, user_id	addItem() add product+qty; removeItem() remove item; clearCart() empty cart; getTotal() compute total price	ERD cart has product_id + quantity per row; class may manage a list internally.
Orders	UI + Backend	order_id, user_id, delivery_id, subtotal, order_date, status	placeOrder() create Orders + OrderDetails from cart; cancelOrder() update status and optionally restore stock	Maps to Orders table; supports admin viewing of orders.
OrderDetails	Backend	order_id, product_id, quantity_id, quantity, unit_price	calculateSubtotal() compute line total (quantity × unit_price)	Matches Order_Details table; separated to support 1 order → many items.
Addresses	UI + Backend	address_id, user_id, name, phone, full_address, is_default	setDefault() set default address; updateAddress() edit address fields	full_address may combine address_line1/2, city, state, etc. from ERD.
Delivery	Backend	delivery_id, address_id, fee, status, estimated_date, delivered_date	updateStatus() update delivery/tracking status	Matches Delivery table; tracking may be basic (prototype).
Payment	Backend	payment_id, order_id, user_id, payment_method, payment_date, total_amount	processPayment() record payment result/confirmation	Matches Payment table; payment can be simulated (no real gateway).
PaymentMethod	UI + Backend	method_id, user_id, method_type, card_type, last_four, expiry_month, expiry_year	validate() basic format checks; remove() delete saved method	Direct mapping; can be optional if payment is simulated.

AIRecommendation	AI + Backend (called by UI)	recommendation_id, user_id, height, weight, preferred_style, recommended_products	generateRecommendation() filter/rank products using profile + tags; getRecommendedProducts() return list for display	Stored separately to keep recommendation history/inputs; recommended_products can be stored as IDs list/JSON.
------------------	--------------------------------	--	--	--

4.3.3 Database Schema

Table: User

Field	Type	Description
user_id	varchar	Primary key
user_name	varchar	User’s full name
user_email	varchar	Email address
user_gender	enum	Male / Female / Other
user_phone	varchar	Contact number
user_profile_pic	varchar	Profile picture URL
user_update_time	timestamp	Last updated time
status	enum	Active / Inactive
role	varchar	User role (e.g., customer/admin)

Table: Category

Field	Type	Description
category_id	varchar	Primary key
category_name	varchar	Clothing category

Table: Product

Field	Type	Description
product_id	varchar	Primary key
category_id	varchar	Foreign key → Category
product_name	varchar	Product name

product_pic1	varchar	Image URL 1
product_pic2	varchar	Image URL 2
product_pic3	varchar	Image URL 3
product_pic4	varchar	Image URL 4
product_description	varchar	Description
product_type	enum	Type (e.g., shirt/pants)
product_price	decimal	Product price
product_status	enum	Active / Inactive

Table: Quantity (Product Variant)

Field	Type	Description
quantity_id	int	Primary key
product_id	varchar	Foreign key → Product
size	enum	S / M / L / XL
product_stock	int	Available stock
product_sold	int	Total sold

Table: Cart

Field	Type	Description
cart_id	varchar	Primary key
user_id	varchar	Foreign key → User
product_id	varchar	Foreign key → Product
quantity	int	Quantity added
added_time	timestamp	Added date/time

Table: Orders

Field	Type	Description
order_id	varchar	Primary key
user_id	varchar	Foreign key → User

delivery_id	varchar	Foreign key → Delivery
order_date	datetime	Order date
orders_status	enum	Pending / Shipped / Delivered
order_subtotal	decimal	Final subtotal

Table: Order_Details

Field	Type	Description
order_id	varchar	Foreign key → Orders
product_id	varchar	Foreign key → Product
quantity_id	int	Foreign key → Quantity
quantity	int	Item quantity
unit_price	decimal	Price per item

Table: Address

Field	Type	Description
address_id	varchar	Primary key
user_id	varchar	Foreign key → User
address_name	varchar	Label (Home/Work)
recipient_name	varchar	Receiver name
phone	varchar	Recipient phone
address_line1	varchar	Address line 1
address_line2	varchar	Address line 2
city	varchar	City
state	varchar	State
post_code	varchar	Postal code
country	varchar	Country
is_default	tinyint	1 = Default address

created_at	timestamp	Creation date
------------	-----------	---------------

Table: Delivery

Field	Type	Description
delivery_id	varchar	Primary key
address_id	varchar	Foreign key → Address
delivery_fee	decimal	Shipping cost
delivery_status	enum	Pending / In Transit / Delivered
estimated_date	date	Expected delivery date
delivered_date	date	Actual delivery date

Table: Payment

Field	Type	Description
payment_id	varchar	Primary key
order_id	varchar	Foreign key → Orders
user_id	varchar	Foreign key → User
tax	decimal	GST / VAT
total_amount	decimal	Final total after tax
payment_method	varchar	Payment type (card/cash/etc.)
payment_status	enum	Paid / Failed
payment_date	datetime	Payment timestamp
discount	decimal	Applied discount

Table: Payment_Method

Field	Type	Description
method_id	varchar	Primary key
user_id	varchar	Foreign key → User
method_type	enum	Card / FPX / E-wallet

card_type	varchar	Visa / Master / Etc
last_four	varchar	Last 4 digits
cardholder_name	varchar	Card holder
expiry_month	int	Expiry MM
expiry_year	int	Expiry YYYY

Table: AIRecommendation

Field	Type	Description
recommendation_id	int	Primary key
user_id	varchar	Foreign key → User
height	float	User height
weight	float	User weight
preferred_style	varchar	Japanese/Korean/European
recommended_products	JSON	AI selected product IDs
created_at	timestamp	Recommendation date

This schema is designed to support the full e-commerce workflow while keeping the database scalable, consistent, and easy to maintain.

Scalability & maintainability

- The catalogue is organised using **Category** → **Product**, so new categories/products can be added without changing application logic.
- Product images are stored as URLs, allowing the system to scale storage using external hosting/CDN if needed.
- **product_status** and **status** fields allow soft-disabling items/users without deleting historical records.

Data integrity (relationships + constraints)

- Foreign keys enforce valid relationships (e.g., Product.category_id, Orders.user_id, Order_Details.product_id), preventing orphan records.
- **Orders** and **Order_Details** are separated to correctly model **one order with many items** and to preserve purchase-time data (unit_price, quantity) even if product prices change later.
- **Quantity** (Product Variant) keeps size-level stock separate from Product to avoid repeating product data for each size and to support accurate stock updates per size.

Normalisation (reducing redundancy)

- The schema follows basic normalisation by splitting repeating or multi-valued data into separate tables:

- Sizes/stock are moved into **Quantity** instead of repeating size columns in Product.
- Multiple purchased items are stored in **Order_Details** instead of repeating product fields in Orders.
- Addresses are stored in **Address** so a user can manage multiple addresses without duplicating user data.

Security (sensitive data handling, even for prototype)

- user_password is stored as a **hashed** value (not plain text).
- Payment is **simulated**, but the schema still avoids storing full card numbers: Payment_Method.last_four only stores the last 4 digits. No CVV is stored.
- All user and payment-related operations should be sent over **HTTPS** (implementation assumption).

4.4 Software Architecture Design

4.4.1 Layered Architecture (revised with links to requirements + clear boundaries)

A **layered architecture** is used to separate concerns between the mobile UI, business logic (including AI), and data storage. This improves maintainability and supports iterative development in the FYP timeline because each layer can be developed and tested independently.

Layer	Main Responsibilities	Linked Functional Requirements	Boundary / Notes
Presentation Layer (Flutter Mobile App)	UI rendering (home, product list/details, cart, profile, recommendations), user interaction handling, basic input validation, state management (Provider/Bloc/GetX), API calls and displaying results	FR1–FR10 (user-facing flows: register/login, browse/search, view details, cart/wishlist, checkout, profile, view recommendations)	Flutter does not run the AI model; it only collects inputs (height/weight/style) and displays the returned recommendations. No sensitive logic stored in the client.
Application Layer (Backend REST API + AI Module)	Authentication + authorization (JWT/session), business rules, product filtering/search, cart/wishlist operations, order creation (simulated payment), admin operations, AI recommendation processing (filter + rank), returns JSON responses	FR1–FR12 (all core logic, including admin and recommendations)	Backend owns: AI inference + recommendation ranking, Supabase Auth JWT validation, and all database writes. Frontend only sends requests with a token.

Data Layer (Supabase PostgreS QL + Storage)	Persistent storage for users/products/inventory/cart/orders/payment(simulated)/delivery/recommendation logs; model file storage (.pkl/.h5); image storage (URL-based)	Supports all FRs by ensuring data can be stored/retrieved consistently	Passwords stored as hashed values; payment is simulated and should not store full card numbers (only metadata like last four digits).
---	---	--	---

Why layered architecture (vs monolithic/microservices)

- Compared to a “single block” monolithic app, this structure makes the system easier to maintain and test (UI changes won’t break DB logic directly).
- Microservices are not suitable for an FYP prototype because they add deployment and communication complexity. A layered backend is simpler but still scalable enough for future extension.

4.4.2 Architecture Diagram

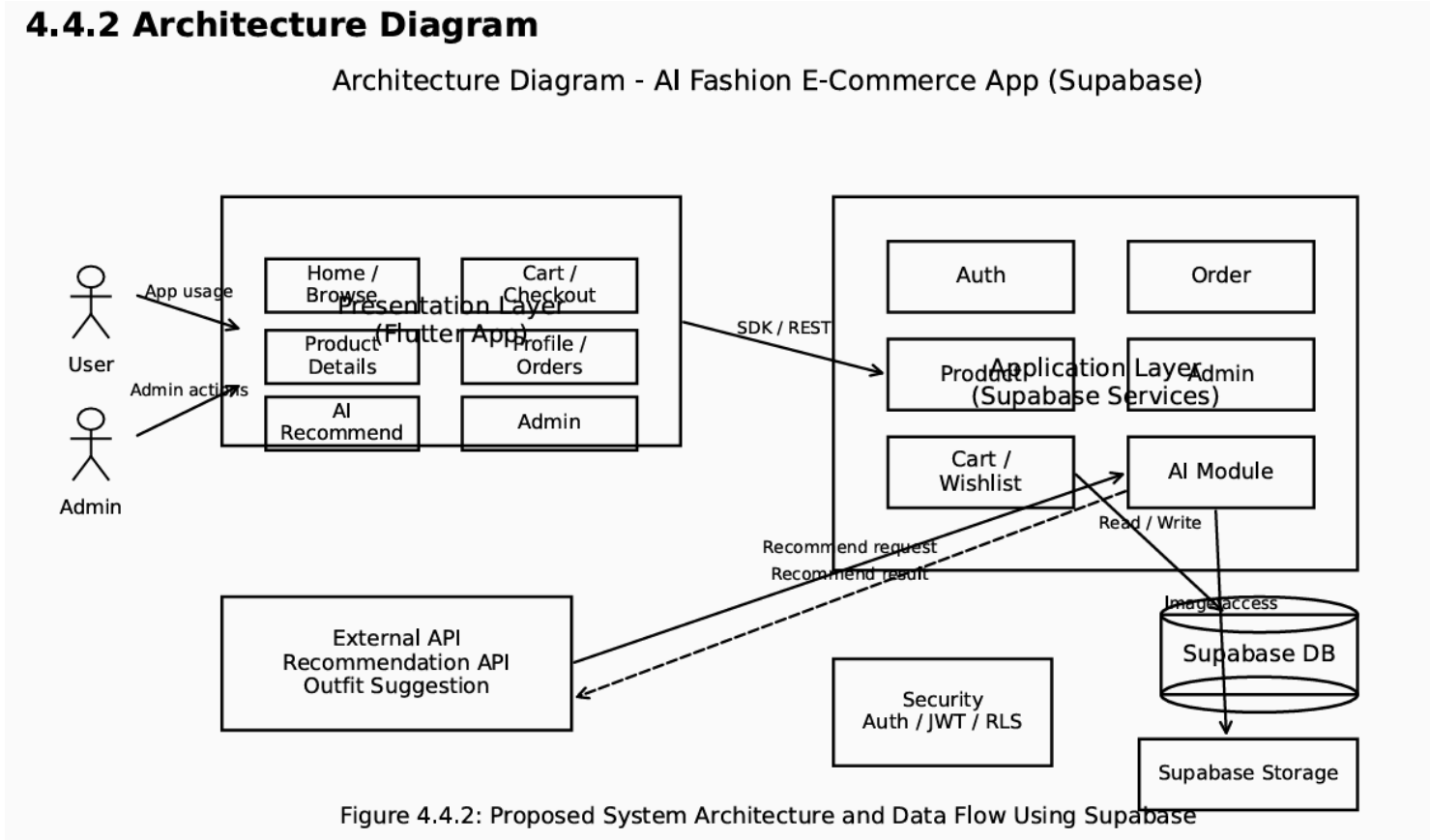


Figure 4.4.2

Architecture Diagram (Description + Data Flow)

Figure 4.4.2 illustrates the high-level architecture of the system using a layered client–server design. The Flutter mobile application (Presentation Layer) communicates with **Supabase services** over HTTPS/SDK calls to perform authentication, catalogue retrieval, cart operations, checkout simulation, and AI recommendation-related data access. Supabase provides the PostgreSQL database, authentication service, and storage layer for product images and related files.

Main data flows shown in the diagram

1. **Authentication flow:** The Flutter app sends login/register requests to the backend. The backend validates credentials, then returns an authentication token/session to the app. Subsequent requests include this token.
2. **Browsing/search flow:** The Flutter app requests product lists or product details. The backend queries the Product/Category/Quantity tables and returns product data as JSON for UI display.
3. **AI recommendation flow:** The user submits height, weight, and style preference from the app. The backend processes these inputs using the AI Recommendation module, retrieves relevant product attributes from the database, generates a ranked recommendation list, stores the result in AIRecommendation, and returns recommended product IDs/details to the app.
4. **Cart and checkout flow:** The app sends add/remove cart actions to the backend. During checkout, the backend creates Orders and Order_Details records and updates stock in Quantity. Payment and delivery are stored as simulated records (no real payment gateway/courier API).
5. **Admin flow:** Admin screens call admin endpoints to manage products and view order activity. The backend performs database updates and returns updated lists.

Assumptions / limitations

- The system assumes internet connectivity because the mobile app relies on the backend API for data and AI processing.
- The backend and database are assumed to be available on a local server or cloud hosting during testing.
- Payment and delivery tracking are implemented at prototype level (simulated), so external payment/courier services are not required.

4.5 Process Design

4.5.1 Login Validation Process (Supabase Auth)

Process steps

1. user enters email and password
2. Flutter app sends sign-in request to Supabase Auth
3. Supabase validates credentials
4. Supabase returns session/JWT
5. app stores session and redirects to Home

Error handling

- **Invalid credentials:** show a generic message (avoid confirming whether an email exists to reduce account enumeration risk).
- **Network failure / timeout:** show “Network error, please try again” and allow retry.
- **Too many failed attempts (if rate-limiting is enabled):** show “Too many attempts, please try again later”.

Supabase Auth is chosen because it improves security and reliability while reducing development time. It provides standard authentication features such as secure password handling, session and JWT management, email/password login, and support for password recovery without requiring the project to build a custom authentication system from scratch. This makes it practical and feasible within the FYP timeframe while remaining well aligned with the project’s PostgreSQL-based backend design.

4.5.2 AI Recommendation Process

Process steps

1. The user inputs:
 - a. **Height**
 - b. **Weight**
 - c. **Preferred style** (e.g., Japanese/Korean/European)
2. The Flutter app sends the inputs to the **AI Model API**.
3. The backend preprocesses the request by:
 - a. validating the input ranges (e.g., height/weight not empty and within expected limits),
 - b. filtering products by the selected **style tag**,
 - c. retrieving product attributes (e.g., category, sizes, stock availability).
4. The AI module applies the recommendation logic/model to compute a **compatibility score** for each candidate product.
5. The system generates a ranked list of recommended products.
6. The backend stores the recommendation session in the AIRecommendation table (user_id, height, weight, preferred_style, recommended_products, timestamp).
7. The backend returns the recommended product list to Flutter.
8. Flutter displays recommendations and allows the user to navigate to Product Details.

Justification of chosen inputs

- **Height and weight** are selected because they are simple measurements that can approximate fit/sizing in a prototype without complex body scanning or many measurement fields.
- **Preferred style** is included because fashion is highly subjective; style tags help increase relevance even with a limited dataset.
- Together, these inputs balance **personalisation** with **low input effort**, supporting usability on mobile.

Limitations / assumptions

- Recommendation accuracy depends on dataset size and quality (e.g., correct style tags and product attributes).
- Height/weight is a simplified body model and may not reflect body shape or personal fit preference (tight/loose).
- If the dataset is small, the recommendations may be less diverse or biased toward frequently available items.

Future improvement The stored recommendation history can later be extended to include user behaviour (clicks, wishlist, purchases) so the system can learn feedback signals and improve ranking using methods such as collaborative filtering or learning-to-rank

4.5.3 Simulated Payment Process

Note: This payment process is **simulated** for the FYP prototype. No real payment gateway integration is implemented.

Process steps

1. The user reviews items in the Cart and taps “Proceed to Payment”.
2. The system validates checkout conditions (cart not empty, address/shipping info provided).
3. The backend performs checkout using a simulated payment:
 - a. create an Orders record (orders_status = "Pending" or "Confirmed"),
 - b. create related Order_Details records for each cart item,
 - c. create a Payment record linked to the order with payment_status = "Paid" (Successful) and payment_date,

- d. update Quantity.product_stock and product_sold.
- 4. The backend returns the order confirmation result to the Flutter app.
- 5. Flutter displays the **Order Successful** screen including:
 - a. Order ID
 - b. Summary of purchased items
 - c. Total amount / payment status

Why payment is simulated (scope justification): Real payment integration is out of scope for an FYP due to additional requirements such as merchant account setup, compliance/security standards (e.g., PCI-DSS), handling real financial transactions, and extensive sandbox-to-production testing. These increase project complexity beyond the limited timeframe.

Why simulation is still useful: Simulated payment still validates the complete system flow end-to-end (cart → checkout → order creation → payment record → order history). This proves that the UI, API, and database schema work correctly together even without real money transfer, enabling functional testing and demonstration during evaluation.

4.6 Algorithm / Model Design

The AI component generates personalized clothing recommendations based on **user body measurements, style preferences, and product attributes**. The proposed solution uses a Hybrid Recommendation Algorithm, combining

- (1) Content-Based Filtering / rule-based scoring to ensure relevant matches and
- (2) **a lightweight ML model** to improve ranking by learning additional patterns (e.g., size compatibility and popularity effects).

4.6.1 Algorithm Overview and Justification

Design Element	Description	Implementation Scope
Algorithm Name	Hybrid Recommendation Algorithm (Content-Based Filtering + ML Scoring)	Implemented in Project 2
Justification	Content-based filtering ensures style-tag relevance and reduces cold-start issues. The ML model re-ranks candidates by learning hidden signals such as approximate size fit and product popularity. Works well for small–medium datasets, which fits FYP constraints.	Conceptual now, trained + evaluated in Project 2
Primary Goal	Recommend products that best match the user’s height, weight, gender, and preferred fashion style.	Implemented in Project 2

FYP feasibility rationale: the system remains functional even if ML performance is limited because the rule-based component can still generate reasonable recommendations. The ML part is lightweight to avoid long training time and heavy compute requirements.

4.6.2 Model Input Features

Feature Type	Features	Reason for inclusion (conceptual justification)
User Features	Height, Weight, Gender, Style preference	Height/weight are low-effort inputs that approximate fit; gender supports filtering where sizing differs; style preference is the strongest signal for fashion relevance and helps reduce search space quickly.
Product Features	Category, Style tags, Clothing type, Available sizes, Price, Popularity score (sales)	These are common, easy-to-store attributes in the database. Popularity improves ranking when user-specific data is limited. Available sizes prevent recommending out-of-stock/irrelevant variants.

Note (limitation): height/weight is a simplified body model; it cannot fully capture body shape or personal fit preference. This is acceptable for a prototype, and can be improved later by adding feedback signals or more measurements.

4.6.3 Model Architecture

Layer	Design	Parameter justification (conceptual)	Implementation Scope
Input	Encoded user + product features	Encodings allow mixed data types (numeric + categorical) to be used together.	Project 2
Hidden Layer 1	32 neurons, ReLU	32 units provide enough capacity to learn non-linear interactions (e.g., style × category × size) while still small enough to train quickly on limited data.	Project 2
Hidden Layer 2	16 neurons, ReLU	Adds depth for feature interaction but keeps the model compact to reduce overfitting.	Project 2
Output	1 neuron, Sigmoid	Produces a 0–1 compatibility probability for ranking.	Project 2

Why not larger/deeper? For an FYP dataset and timeline, larger networks increase overfitting risk and training complexity without guaranteed benefit.

4.6.4 Activation & Loss Functions

Component	Choice	Justification	Implementation Scope
Hidden activation	ReLU	Fast training, stable gradients, widely used for small neural nets.	Project 2
Output activation	Sigmoid	Converts output into a probability-like score for ranking/thresholding.	Project 2
Loss function	Binary Cross-Entropy	Suitable when the training label is “compatible / not compatible” or derived from implicit feedback (click/add-to-cart).	Project 2
Optimizer	Adam	Efficient default optimizer; works well without extensive hyperparameter tuning.	Project 2

4.6.5 Compatibility Scoring Formula

Before ML scoring, the system computes a baseline score to ensure relevance and to support recommendations even if the trained model is weak:

CompatibilityScore=w1(StyleMatch)+w2(SizeFit)+w3(Popularity)

Where:

- **StyleMatch** = 1 if style matches, else 0–0.5 (partial match if multiple style tags exist)
- **SizeFit** = estimated fit using BMI approximation and available sizes
- **Popularity** = normalised sales count (e.g., based on product_sold)

Weight	Value	Justification (conceptual)
(w_1) StyleMatch	0.5	Style is the strongest determinant of user preference, so it gets the highest weight.
(w_2) SizeFit	0.3	Fit matters in clothing purchase decisions, but BMI-based fit is approximate, so weight is moderate.
(w_3) Popularity	0.2	Helps for cold-start and ties, but popularity should not dominate personal preference.

Scope: the formula is **implementable in Project 2** and provides a strong baseline for evaluation.

4.6.6 Final Prediction Equation

The final ranking score combines the rule-based score and the ML model output:

FinalScore=0.6(CompatibilityScore)+0.4(MLProbability)

Parameter	Value	Why this split is reasonable for FYP
Rule-based weight	0.6	More reliable when training data is limited; guarantees relevance and prevents unreasonable outputs.
ML weight	0.4	Allows learning-based improvement without making the system dependent on a perfect model.

Implementation note (Project 2): the 0.6/0.4 split is an initial design choice and will be tuned based on validation results (e.g., user testing or offline metrics such as Precision@K).

4.7 Security Design

This security design addresses the main risks in a mobile e-commerce system: unauthorised access, data leakage during transmission, misuse of user accounts, and exposure of sensitive information such as credentials and payment-related data.

The approach combines trusted third-party security services provided by **Supabase Auth** with backend/API protections implemented by the system.

4.7.1 Security Measures, Risk Addressed, and Ownership

Area	Design Approach	Risk Addressed	Handled By
Authentication	Supabase Auth (email/password login)	Prevents unauthorised logins and reduces risk of insecure custom authentication implementation	Third-party (Supabase)
Password Storage	Supabase-managed secure password hashing	Prevents password leakage from database exposure	Third-party (Supabase)
Session Handling	Token-based authentication (Supabase JWT/session) with expiry	Reduces risk of session hijacking and long-lived tokens	Supabase + System (system verifies token on API)
Secure Communication	HTTPS for API requests/data transfer	Prevents eavesdropping / man-in-the-middle attacks	System configuration (hosting/SSL)
Database Access Control	Role-based access control (Admin vs User)	Prevents unauthorised product/order management and privilege escalation	System (Backend)
Data Privacy	Minimal data collection; height/weight optional; no raw PII in AI dataset	Reduces impact if dataset is leaked; improves privacy compliance	System (Design choice)
AI Training Data Ethics	Anonymise identifiers; do not train on raw personal data	Prevents re-identification and unethical use of sensitive info	System (AI pipeline)

Local Storage Security	Store tokens using secure Flutter storage (Keychain/Keystore)	Prevents token theft from device storage	System (Mobile app)
Input Validation	Backend validates inputs; parameterised queries/ORM	Prevents SQL injection, malformed requests, and data corruption	System (Backend)
Payment Data Protection	Payment is simulated; no full card number/CVV stored	Reduces exposure of financial data and compliance burden	System (Scope decision)

Clarification note: Supabase Auth secures credential handling and token generation, but the backend must still enforce authorisation, for example by checking the token and role before allowing admin endpoints. In other words, Supabase authenticates who the user is, while the system controls what the user is allowed to do.

4.7.2 Security Validation / Testing Plan (Project 2)

Security will be validated in Project 2 through:

- Authentication testing: verify login/logout, token expiry handling, invalid token rejection.
- Authorization testing: attempt access to admin APIs using a normal user token (should be denied).
- Input validation testing: submit invalid/malicious inputs (e.g., long strings, special characters) to ensure the API rejects them safely.
- Transport verification: confirm HTTPS is enforced and no sensitive data is sent via plain HTTP.
- Data handling review: confirm no plain-text passwords, no full payment details stored, and recommendation logs do not expose PII beyond user_id.

4.8 Chapter Summary and Evaluation

This chapter presented the complete system design, including the workflow and UI structure, the layered software architecture, the ERD/class/database schema, the AI recommendation model design, and the security mechanisms. The proposed architecture separates responsibilities across the **Flutter presentation layer**, **backend application layer**, and **SQL data layer**, ensuring the system is organised and easier to maintain during development.

From an evaluation perspective, the design is **suitable** and **feasible** for an FYP because it supports all core e-commerce requirements (browse/search, product details, cart, checkout, order history) while integrating the AI recommendation module without disrupting the normal shopping flow. The database design is normalised (e.g., Orders vs Order_Details, Product vs Quantity) and uses clear relationships to improve **data integrity**, reduce redundancy, and allow future expansion such as adding more product variants, additional payment methods, or richer tracking status. The layered architecture also supports scalability because the backend can be extended independently of the mobile app, and data storage can grow without changing the UI logic.

A key design trade-off is the use of a **lightweight hybrid recommendation model** rather than a complex deep learning approach. This choice is intentional: lightweight models are faster to train and deploy, require less data, and are more achievable within the project timeline, while still demonstrating meaningful personalisation through style filtering and compatibility scoring. Similarly, payment and delivery are modelled in the schema but implemented as **simulated processes**, which allows end-to-end workflow validation without the complexity and compliance requirements of real payment gateway integration.

This design will directly guide the next chapter by defining (1) the API endpoints and module responsibilities for implementation, (2) the database tables and relationships for CRUD operations, and (3) the test targets for validation—such as authentication accuracy, recommendation response time, order creation correctness, and role-based access control. The next chapter will therefore focus on implementing the modules according to this design and performing functional and performance testing to verify that the system behaves as specified.

Chapter 5: Implementation and Testing

5.1 Implementation

5.1.1 Development of Mobile Application

The mobile application was developed using Flutter, following the presentation-layer design proposed in Chapter 4. Flutter was selected because it supports rapid user interface development, reusable widgets, and efficient integration with backend services. The application was organised into modular components so that screens, models, services, and database access logic were separated clearly. This structure improved maintainability and simplified debugging during development.

From the user perspective, the implemented prototype follows a standard fashion shopping flow. After launching the application, the user can access the main product discovery features through the home interface. Product discovery was implemented through category-based browsing and keyword search. To better support the recommendation objective of the project, the product catalogue was organised around a smaller set of outfit-oriented categories rather than overly detailed retail categories. This made it easier to support outfit generation because products could be matched as tops, bottoms, outerwear, dresses, and accessories.

The authentication function was integrated using Supabase Auth. Users can register and log in using email and password, and the application stores the authenticated session so that protected features such as profile management, cart, wishlist, and order-related records can be linked to the correct user account. Compared with implementing a custom authentication module, this approach reduced development complexity while still supporting secure session handling and user identity management. This is consistent with the security and process design already defined in the earlier chapters.

The product browsing module was connected to the PostgreSQL database through Supabase queries. Product information such as product name, description, price, category, size availability, and image path can be retrieved from the database and displayed in the mobile interface. Since not all product images were ready during the current implementation stage, the product image fields were intentionally left empty for some records, and placeholder rendering was used where necessary. This allowed the product listing screens to be tested even before the final image assets were prepared.

The user profile module was implemented to collect and store the inputs required by the recommendation logic. In this project, the main user profile inputs are height, weight, and preferred fashion style. These values are stored in the user profile table and are used by the recommendation function when the user requests outfit suggestions. The profile management screen allows the user to update these values so that the recommendation results can reflect changes in user preference or body-related input.

The wishlist and cart modules were implemented as separate database-backed features. The wishlist allows users to save products for later review, while the cart supports temporary shopping decisions before checkout. In the cart module, the user can add items, update quantity, and remove items. These actions were linked to the database so that the cart reflects the current user session and can later be transformed into an order record during checkout.

The checkout module was implemented as a simulated payment process rather than a real payment gateway. This implementation decision follows the design justification in Chapter 4, where real payment integration was considered out of

scope due to compliance, security, and third-party service complexity. Instead, the prototype records order, order detail, payment, and delivery data in the database, allowing the complete purchase flow to be demonstrated without handling real financial transactions.

The AI recommendation feature was implemented as the key differentiating function of the application. The recommendation flow accepts the user's height, weight, and preferred style as inputs. It then filters and ranks candidate products using the hybrid recommendation approach defined in the design chapter. In practical terms, the prototype uses profile-based filtering and structured product metadata to produce a recommendation list that is more relevant than generic catalogue browsing. The generated recommendations are displayed in a dedicated recommendation section and can be opened using the same product detail flow as manually browsed items. This ensures that AI recommendation is integrated into the application without requiring a separate purchase workflow.

5.1.2 Backend and Database Integration

The application backend relied primarily on Supabase services. Supabase PostgreSQL was used to store user profiles, categories, products, product quantities, wishlists, cart items, addresses, orders, payments, deliveries, and recommendation logs. The database schema was normalised so that repeated data was separated appropriately, such as product size stock being stored in a quantity table and purchased items being stored in order details. This reduced redundancy and kept the application structure aligned with the design objectives defined earlier.

Row Level Security policies were enabled so that users could only access and modify their own personal data. Public product and category data were made readable to support the product browsing flow, while user-owned tables such as cart, wishlist, payment methods, and recommendation records were restricted to authenticated users. A trigger-based profile creation mechanism was also used so that when a user account is created through Supabase Auth, a corresponding profile row can be inserted into the application database automatically.

Storage support was planned for product image handling. Even though image fields were left empty during the current seed-data stage, the database and frontend structure were prepared so that image URLs or storage paths can later be added without changing the overall application logic.

5.3 Testing Strategies and Approaches

The testing phase was designed to verify both the correctness of the implemented features and the suitability of the system as a mobile fashion shopping prototype. The testing strategy covered four main aspects: functional testing, integration testing, usability-oriented checking, and basic performance observation.

Functional testing was used to confirm that each user-facing module works according to the requirements. This included registration and login, viewing products, viewing product details, managing profile data, adding or removing wishlist items, updating cart contents, generating recommendations, and completing simulated checkout.

Integration testing was important because the application depends on communication between the Flutter frontend and Supabase services. Therefore, test scenarios were designed to confirm that data written through the mobile app is stored correctly in the database, and that changes in the database can be retrieved and displayed correctly in the UI. Examples include cart updates, profile updates, and order record creation.

Usability checking was also considered because the system is intended for mobile-first interaction. Since users must be able to move from browsing to cart and checkout with minimal confusion, screen transitions, button placement, and information clarity were reviewed during testing. In particular, the recommendation feature was checked to ensure that it did not interrupt the normal shopping flow.

Basic performance observation was used for important user actions, especially recommendation generation and product loading. Since the project uses a lightweight recommendation design rather than a large deep learning model, the expectation was that recommendation results should appear within a short and practical time frame for prototype use. This is aligned with the project's non-functional requirement that recommendation responses should remain fast enough for mobile use.

5.4 Test Plan

The purpose of the test plan was to ensure that the implemented system satisfies the key functional requirements and remains stable enough for demonstration and evaluation. Testing was conducted after each major implementation increment, following the Agile development approach stated earlier in the report. This allowed errors to be detected early and reduced the risk of late-stage integration problems.

The scope of testing covered the following modules:

1. user registration and login
2. product list and product details
3. user profile management
4. AI recommendation
5. wishlist
6. cart
7. simulated checkout and order creation

The main expected outputs from testing were:

1. correct database read/write behaviour
2. correct UI behaviour for common user actions
3. correct recommendation generation under predefined profile scenarios
4. stable end-to-end flow from browsing to order confirmation

5.5 Test Data

Representative test data were prepared to verify both normal and edge-case behaviour. For authentication testing, valid and invalid email-password combinations were used. For profile-based recommendation testing, several combinations of height, weight, and style preference were used to ensure that the system could produce results for multiple user types.

Sample recommendation test profiles included:

- height: 160 cm, weight: 50 kg, style: Korean
- height: 170 cm, weight: 65 kg, style: Japanese
- height: 180 cm, weight: 78 kg, style: European

For product data, seeded records from the database were used. These records covered five main categories: Top, Bottom, Outerwear, Dress, and Accessories. Product names were intentionally kept short and simple so that they could be displayed clearly in a mobile card layout. Size-level stock data were also prepared to test whether the recommendation and purchase flows remained compatible with stock-aware filtering.

For checkout testing, sample address data, simulated payment method metadata, and cart quantities were used. Since payment is simulated in this prototype, test data focused on whether records were created correctly rather than on real payment gateway outcomes.

5.6 Test Cases

Table 5.6 Test Case 1

Test Case: 1			Test Case Name: User Registration			
System: AI-Powered Clothing App for Personalized Outfit Recommendation			Subsystem: Account & Authentication			
Designed by: Sia Jin Sheng			Design Date: 12-03-2026			
Executed by: Sia Jin Sheng			Execution Date: 13-03-2026			
Short Description: Verify that a new user can register successfully using email and password.						
Precondition: User is not logged in and the email address has not been registered in the system.						
Step	Action	Test Data	Expected System Response	Actual System Response	Pass/Fail	Comments
1	Launch app and tap Sign Up	-	Registration screen is displayed	Same as Expected	Pass	
2	Enter registration details	Name: Jin Sheng Email: testuser01@gmail.com Password: test1234 Confirm Password: test1234	Input is accepted and no validation error is shown	Same as Expected	Pass	
3	Tap Register	-	System creates account through Supabase Auth and	Same as Expected	Pass	

Table 5.6 Test Case 3

Test Case: 3			Test Case Name: Product Browsing and Product Details			
System: AI-Powered Clothing App for Personalized Outfit Recommendation			Subsystem: Product Catalogue			
Designed by: Sia Jin Sheng			Design Date: 12-03-2026			
Executed by: Sia Jin Sheng			Execution Date: 13-03-2026			
Short Description: Verify that the user can browse products by category and open a product details page.						
Precondition: Product data already exists in the Supabase database.						
Step	Action	Test Data	Expected System Response	Actual System Response	Pass/Fail	Comments
1	Launch app and open Home screen	-	System filters and shows products under the selected category	Same as Expected	Pass	
2	Select a category	Category: Top	System filters and shows products under the selected category	Same as Expected	Pass	
3	Review product information	Tap a product card	Product details page is displayed	Same as Expected	Pass	
4	Review product information	-	Product name, price, description, and available data are shown correctly	Same as Expected	Pass	
Post Condition: User can browse the catalogue and access detailed product information.						

Table 5.6 Test Case 4

Test Case: 4			Test Case Name: Update User Profile			
System: AI-Powered Clothing App for Personalized Outfit Recommendation			Subsystem: User Profile			
Designed by: Sia Jin Sheng			Design Date: 12-03-2026			
Executed by: Sia Jin Sheng			Execution Date: 13-03-2026			
Short Description: Verify that the user can update profile data used by the recommendation module.						
Precondition: User is logged in.						
Step	Action	Test Data	Expected System Response	Actual System Response	Pass/Fail	Comments
1	Open Profile screen	-	Profile page is displayed	Same as Expected	Pass	
2	Enter or edit profile information	Height: 170 Weight: 65 Style: Korean	Input is accepted without validation error	Same as Expected	Pass	

Step	Action	Test Data	Expected System Response	Actual System Response	Pass/Fail	Comments
1	Open product details page	Product: Knit Top	Product details screen is displayed	Same as Expected	Pass	
2	Tap Add to Wishlist	-	Product is added to wishlist successfully	Same as Expected	Pass	
3	Open Wishlist page	-	Added product appears in wishlist list	Same as Expected	Pass	
4	Remove product from wishlist	-	Product is removed from wishlist and no longer shown	Same as Expected	Pass	
Post Condition: A new account is created in the authentication system and linked to the user profile record.						

Table 5.6 Test Case 7

Test Case: 7			Test Case Name: Cart Management and Simulated Checkout			
System: AI-Powered Clothing App for Personalized Outfit Recommendation			Subsystem: Cart & Order Processing			
Designed by: Sia Jin Sheng			Design Date: 12-03-2026			
Executed by: Sia Jin Sheng			Execution Date: 13-03-2026			
Short Description: Verify that the user can add an item to cart, modify quantity, and place an order through simulated checkout.						
Precondition: User is logged in, product stock is available, and user has a valid shipping address.						
Step	Action	Test Data	Expected System Response	Actual System Response	Pass/Fail	Comments
1	Open product details page and tap Add to Cart	Product: Basic Tee Size: M Quantity: 1	Product is added to cart	Same as Expected	Pass	
2	Open Cart page	-	Cart displays selected item and subtotal	Same as Expected	Pass	
3	Update quantity	Quantity: 2	Cart total is updated correctly	Same as Expected	Pass	
4	Tap Proceed to Checkout	-	Checkout page is displayed	Same as Expected	Pass	
5	Enter / select shipping and payment details	Address: Home Payment: Simulated Card	Input is accepted and checkout can continue			
6	Tap Place Order / Confirm Order	-	System creates order, order detail,			

			payment, and delivery records			
7	View confirmation page	-	Order confirmation and summary are displayed			

Post Condition: Order is created successfully and checkout flow is completed.

Table 5.6 Test Case 8

Test Case: 8			Test Case Name: Invalid Login			
System: AI-Powered Clothing App for Personalized Outfit Recommendation			Subsystem: Account & Authentication			
Designed by: Sia Jin Sheng			Design Date: 12-03-2026			
Executed by: Sia Jin Sheng			Execution Date: 13-03-2026			
Short Description: Verify that the system rejects login when invalid credentials are entered.						
Precondition: Login screen is accessible.						
Step	Action	Test Data	Expected System Response	Actual System Response	Pass/Fail	Comments
1	Open Login screen	-	Login page is displayed	Same as Expected	Pass	
2	Enter invalid credentials	Email: testuser01@gmail.com Password: wrong123	Input is accepted	Same as Expected	Pass	
3	Tap Login	-	System rejects login and displays an error message	Same as Expected	Pass	
4	Observe system state	-	User remains on login screen and no session is created	Same as Expected	Pass	
Post Condition: Invalid login attempt is blocked successfully.						

5.7 Testing Results and Evaluation

Overall, the implemented prototype achieved the main implementation goals of the project. Core user flows such as authentication, product browsing, profile management, cart/wishlist actions, and simulated checkout could be executed successfully in the integrated environment. The Supabase connection was validated through direct product table queries, confirming that the Flutter application could read data from the backend correctly. This provided confidence that the main data path of the system was functioning as intended.

From a recommendation perspective, the prototype successfully demonstrated the feasibility of generating personalised outputs from limited structured user input. While the recommendation logic is lightweight and not intended to match the sophistication of large-scale commercial systems, it was sufficient to show fit-aware and style-aware behaviour within the FYP scope. This is consistent with the project's original intention to use a practical hybrid recommendation design rather than a complex large-data model.

The main implementation limitations observed during testing were related to prototype scope rather than system failure. Product image coverage was incomplete because final image assets had not yet been added for all seeded products. In addition, payment remained simulated, and delivery tracking remained status-based rather than integrated with a real courier service. However, these limitations were already anticipated in the system design and do not prevent the prototype from demonstrating the complete end-to-end shopping flow.

5.8 Chapter Summary and Evaluation

This chapter presented the implementation and testing of the AI-powered clothing mobile application. The implementation phase translated the earlier design into a working Flutter prototype integrated with Supabase Auth, PostgreSQL storage, and secure access control. The developed modules support the main fashion e-commerce flow while incorporating AI recommendation as the project's key differentiating feature.

From an evaluation perspective, the implementation is suitable for the project scope because it demonstrates the main technical objectives in a realistic but manageable form. The system is able to support product browsing, profile-based recommendation generation, wishlist and cart interaction, and simulated order creation within a single mobile workflow. Testing indicates that the prototype is functionally workable and technically feasible as an FYP solution.

At the same time, the chapter also confirms that some aspects remain prototype-level by design. These include simulated payment, limited delivery tracking, and incomplete product image assets. These limitations do not invalidate the implementation; rather, they define the practical boundary of the current system version and provide a clear direction for future enhancement in the concluding chapter.